

Deployment-Oriented Neural Network Methods For Battery Management Systems:

Ageing Estimation, Robustness,
Embedded Optimization, and Hardware
Integration

submitted by
M. Sc.
Florian Rzepka
born in Berlin

Faculty IV – Electrical Engineering and Computer Science
Technische Universität Berlin
in partial fulfillment of the requirements for the academic degree of

Doctor of Engineering
– Dr.-Ing. –
Dissertation

Doctoral Committee:
Chair: tbd
Reviewer: tbd

Reviewer: tbd

Date of the scientific defense: tbd

Berlin, 2026

Preface

This dissertation investigates deployment-oriented artificial intelligence for battery management systems and combines model development, robustness analysis, embedded implementation, and hardware integration within one common research framework.

Abstract

This dissertation develops a deployment-oriented perspective on artificial intelligence for battery management systems. The work begins with neural-network-based ageing estimation for stationary storage systems, continues with robustness benchmarking of embedded battery state estimation under disturbed measurements, and then addresses the transfer of SOC and SOH models to microcontrollers using pruning and quantization. A comparative analysis across model architectures extends the compression perspective, while a dedicated hardware chapter documents the custom battery management system platform and the practical route from algorithm development to embedded execution. The central result is that battery intelligence must be evaluated not only by nominal prediction accuracy, but against a combined set of battery-management requirements across performance, hardware, and deployment.

Executive Summary

This dissertation is structured as a continuous, application-oriented account of artificial intelligence in battery management. Its main focus lies on neural architectures for ageing estimation in stationary storage systems, the robustness of embedded state estimation under realistic measurement disturbances, the efficient execution of such models on microcontrollers through pruning and quantization, and their integration into a real hardware and BMS platform. The overarching idea is that AI-enabled BMS methods should be evaluated not only by accuracy, but along the three requirement dimensions of performance, hardware, and deployment.

Table of Contents

Preface	III
Abstract	V
Executive Summary	VII
Abbreviations	XIII
Symbols	XV
List of Figures	XX
List of Tables	XXI
1 Introduction and Motivation	1
1.1 Research Objective and Requirement Dimensions	1
1.2 Scientific Contributions	2
1.3 Specific Research Objectives	3
1.4 Structure of the Dissertation and Development Path	4
2 Battery-Management Requirement Framework	7
2.1 Performance Requirements	8
2.2 Hardware Requirements	9
2.3 Deployment Requirements	9
2.4 Coverage of the Framework in the Dissertation	9
3 State of the Art and Technical Background	11
3.1 Battery Management Systems	11
3.2 Neural Networks for Battery Ageing and State Estimation	11
3.3 Embedded AI in Battery Applications	11

3.4	Compression and Model-Hardware Co-Design	12
4	Experimental Basis and Hardware Ecosystem	13
4.1	Data Sources and Ageing Campaigns	13
4.2	Embedded Targets and Execution Chain	14
4.3	Custom BMS-STEP Platform	15
4.3.1	System Objective and Laboratory Scope	16
4.3.2	Hardware Architecture	17
4.3.3	Firmware and Execution Concept	18
4.3.4	Experimental Relevance for Embedded AI	19
5	Neural Network Architecture for Ageing Estimation in Stationary Storage Systems	21
5.1	Application Context	21
5.2	Raw Cell Dataset, Parameters, and Measured Quantities	22
5.3	Preprocessing Method and Feature Engineering	23
5.3.1	Cumulative Current	23
5.3.2	State-of-Health Reference	24
5.3.3	Correlation Analysis	25
5.3.4	Formation of the Lag Sequence	26
5.3.5	Training, Validation, and Test Splits	27
5.3.6	Data Scaling	29
5.4	Multilayer Perceptron Model Structure	30
5.5	Evaluation and Hyperparameter Tuning	31
5.6	Results and Performance Evaluation	32
5.7	Interpretation of the Ageing-Estimation Results	35
6	Robustness Benchmarking of Embedded Battery State Estimation	37
6.1	Application Context and Objective	37
6.2	State of the Art and Benchmark Gap	38
6.3	Estimator Classes Under Test	39
6.4	Embedded-Oriented Model Preparation	40
6.5	Measurement-Only Benchmark Principle	41
6.6	Disturbance Taxonomy and Scenario Design	41
6.7	Metrics and Evaluation Logic	43
6.8	Dataset Basis and Online Execution Setup	44
6.9	Baseline Performance	45

6.10	Current-Bias and Noise Sensitivity	46
6.11	Initialization Mismatch and Recovery	49
6.12	Missing Samples, Timing Faults, and Signal Integrity	50
6.13	Voltage-Spike Sensitivity and Cross-Scenario Comparison	53
6.14	Interpretation of the Robustness Results	55
6.15	Decision-Oriented Deployment Interpretation	56
6.16	Limitations and Future Extensions	57
7	Embedded Artificial Intelligence for SOC and SOH Estimation on Microcontrollers	59
7.1	Embedded Deployment Objective	59
7.2	Dataset Basis and Experimental Scope	60
7.3	Feature Engineering and Reference Targets	62
7.4	LSTM Architecture and Training Setup	63
7.5	Structured Pruning for MCU Deployment	64
7.6	INT8 Quantization and Manual STM32 Export	65
7.7	Benchmark Methodology and KPI Logic	66
7.8	Streaming Prediction Accuracy	67
7.9	Error Distribution and Local Transient Behaviour	69
7.10	Resource Efficiency, Latency, and Trade-Off Summary	72
7.11	Implications for MCU-Based BMS	74
8	Architecture-Dependent Effects of Pruning and Quantization	75
8.1	Problem Statement	75
8.2	Comparison Framework	75
8.3	Scientific Contribution	77
9	General Discussion and Cross-Paper Synthesis	79
9.1	Synthesis of the Main Findings	79
9.2	Synthesis Matrix	80
9.3	Scientific Overall Contribution	80
9.4	Methodological Strengths and Limitations Across the Studies	81
9.5	Positioning Within the State of Research	82
9.6	Technical Implications for AI-Based BMS	82
9.7	Integrated Design Guidelines	83

10 Conclusion and Outlook	85
10.1 Overall Conclusion	85
10.2 Boundary Conditions and Remaining Gaps	85
10.3 Outlook	86
Literatur	87
A Supplementary Material	89

Abbreviations

AI	Artificial Intelligence
BMS	Battery Management System
ECM	Equivalent Circuit Model
LSTM	Long Short-Term Memory
MCU	Microcontroller Unit
MLP	Multilayer Perceptron
SOC	State of Charge
SOH	State of Health

Symbols

SOC	State of charge
SOH	State of health
Q	Cumulative charge or charge throughput
t	Time
U	Voltage
I	Current

List of Figures

1.1	Roadmap of the overall work. The study chapters are embedded in a shared technical basis and are connected through a final synthesis toward deployment-oriented BMS intelligence.	6
2.1	Battery-management requirement framework used throughout the dissertation. The framework groups deployment-oriented BMS requirements into performance, hardware, and deployment aspects and relates estimator quality to embedded feasibility and system realization.	8
4.1	Design-of-experiments view of the laboratory ageing and operating-condition space used to structure the embedded battery test campaigns and the resulting datasets.	14
4.2	Board-level view of the custom BMS-STEP platform with integrated sensing, controller, communication, and auxiliary interfaces.	17
4.3	Reduced system-level block view of the BMS-STEP platform showing the interaction between current sensing, BMS front-end, and microcontroller domain.	18
5.1	SOH progression of the 14 cells over equivalent full cycles and over absolute time, transferred from the MDPI paper.	25
5.2	Correlation heatmap used in the paper to identify direct linear relations between SOH and candidate input variables.	26
5.3	Lag-sequence construction used to transform raw cell measurements into supervised MLP samples.	27
5.4	Processing and evaluation workflow of the MLP-based SOH estimator, transferred from the MDPI paper.	29
5.5	MLP architecture used in the MDPI paper for SOH estimation.	31

5.6	MAE matrix from the MDPI paper showing the interaction between sequence length and temporal resolution.	33
5.7	Comparison of predicted and measured SOH trajectories for three representative test cells.	34
5.8	Predicted versus measured SOH values for the representative test cells used in the MDPI paper.	34
6.1	Taxonomy of the disturbance scenarios used in the robustness benchmark, grouped into input disturbances, initialization errors, and signal-integrity faults.	43
6.2	Baseline performance on the selected full-cell benchmark trajectory. The hybrid direct-measurement model provides the lowest clean-condition error, whereas the direct-measurement model shows the largest baseline deviation.	46
6.3	Sensitivity to current-measurement bias, including a representative disturbed window, the bias sweep, and the resulting SOC divergence over time.	48
6.4	Noise sensitivity under additive current noise, including a representative disturbed window, global MAE penalties, and local output-variability effects.	49
6.5	Recovery after initialization mismatch in a reset-free evaluation window, showing the disturbed SOC trajectories and the corresponding absolute SOC error over time.	50
6.6	Signal-integrity summary comparing the global MAE penalties for missing samples, irregular sampling, and burst dropout together with the burst-dropout recovery behavior.	51
6.7	Local transition into the burst-dropout interval. The figure shows how the estimators behave while the input stream is frozen.	52
6.8	Post-gap recovery after burst dropout, including the disturbed SOC trajectories and the corresponding absolute SOC error after the frozen interval.	53
6.9	Voltage-spike sensitivity, including a representative local response, the spike-severity summary, and the aligned post-spike excess-error behavior.	54

6.10	Cross-scenario error increase relative to the baseline case. The heatmap condenses the disturbed-scenario penalties and makes estimator-specific ranking shifts visible across disturbance classes.	55
6.11	Methodology overview of the robustness benchmark, including disturbance injection, matched estimator evaluation, and comparison logic.	57
6.12	Decision-oriented synthesis of the robustness benchmark showing how estimator-selection guidance can be derived from disturbance-specific results.	58
7.1	Design-of-experiments cube of the LFP ageing dataset used in the embedded AI paper.	61
7.2	Representative SOC trajectory showing cyclic ageing, check-up blocks, and stationary-storage load-profile segments.	61
7.3	SOH progression across the LFP ageing campaign, illustrating the broad spread between cells and operating points.	62
7.4	Stateful LSTM + MLP structure used for both SOC and SOH estimation in the embedded paper.	63
7.5	End-to-end deployment pipeline from dataset and PyTorch training to header export and STM32 benchmarking.	64
7.6	Structured pruning concept used in the embedded AI paper.	65
7.7	Per-row INT8 quantization and export concept used for the STM32 deployment.	66
7.8	Streaming SOC dashboard from the embedded AI paper for base, pruned, and quantized deployment.	68
7.9	Streaming SOH dashboard with causal filtering for the three deployment variants.	69
7.10	SOC error distribution from the embedded paper, including MAE boxplots and error histogram.	70
7.11	SOH error distribution from the embedded paper, including MAE boxplots and error histogram.	70
7.12	Local pulse-region analysis of the embedded SOC estimator.	71
7.13	Local check-up-region analysis of the embedded SOC estimator.	71
7.14	Resource landscape of SOC and SOH variants, including parameter count, estimated flash, measured flash, and RAM.	73
7.15	Latency distributions for SOC and SOH variants on the STM32H753ZI.	73

8.1	Quantization workflow used as the methodological starting point for the cross-architecture compression study.	77
-----	--	----

List of Tables

1.1	Cumulative structure of the dissertation and function of the core contribution chapters within the overall argument.	5
5.1	Ageing scenarios of the stationary-storage dataset transferred from the MDPI paper.	23
5.2	Tuned MLP configuration transferred from the MDPI paper.	31
7.1	STM32 engineering KPIs for the SOC deployments transferred from the embedded AI paper.	67
7.2	STM32 engineering KPIs for the SOH deployments transferred from the embedded AI paper.	67
7.3	Composite pruning/quantization summary from the embedded AI paper. Values are reported as SOC/SOH.	74
8.1	Experiment matrix for analysing architecture-dependent pruning and quantization effects under matched embedded conditions.	76
9.1	Synthesis matrix linking the main contribution chapters to the three central requirement dimensions.	80

Introduction and Motivation

Battery management systems require reliable internal-state information to operate lithium-ion batteries safely, efficiently, and over long service intervals. Estimates of state of charge and state of health influence charging control, power limitation, lifetime-aware operation, and higher-level energy-management decisions. At the same time, battery behaviour becomes increasingly nonlinear once ageing, operating-point variation, and real measurement imperfections are taken into account. This creates a strong motivation for AI-based estimation methods that can capture complex relations beyond compact analytical descriptions.

Practical BMS deployment, however, imposes requirements that go beyond nominal prediction accuracy under controlled evaluation settings. Estimators must remain useful under disturbed measurements, limited computation and memory, causal execution, and real firmware and sensing constraints. These requirements become especially relevant in decentralized stationary-storage systems and remote microgrid-like applications, where battery supervision must remain locally available over long maintenance intervals and without permanent cloud or internet connectivity. The central scientific gap addressed in this dissertation is therefore not the absence of accurate battery-AI models as such, but the limited availability of a consistent evaluation and development framework that brings model quality, robustness, embedded feasibility, and hardware-level realization together.

1.1 Research Objective and Requirement Dimensions

The research objective of the dissertation is to develop and validate deployment-oriented neural-network methods for battery management systems along three linked requirement dimensions:

- The first requirement dimension addresses estimator performance. This includes not only nominal accuracy under controlled conditions, but also robustness

against disturbed measurements, recovery behaviour after state inconsistency, and stable operation under realistic signal corruption and imperfect online conditions.

- The second dimension concerns hardware requirements. Battery-AI methods intended for practical BMS use must be judged against the limits of the target platform, including memory demand, runtime latency, integration constraints, and restrictions imposed by the measurement chain itself.
- The third dimension focuses on deployment requirements from model concept to embedded realization. This includes feature engineering, architecture selection, compression, model-to-device transfer, adaptability, scalability, and the firmware-level steps required to turn a trained estimator into a deployable BMS component.

These three dimensions are treated as interdependent rather than separate evaluation layers. The central argument of the dissertation is that battery-AI methods become practically meaningful only when predictive quality, embedded realizability, and hardware-level integration are developed and assessed together.

1.2 Scientific Contributions

- At the conceptual level, the dissertation establishes a unified deployment-oriented framework for evaluating battery-AI methods across performance, hardware, and deployment requirements. This framework provides the common perspective through which the subsequent studies are interpreted rather than treating accuracy, robustness, and embedded realization as isolated concerns.
- For ageing estimation in stationary-storage cells, the work introduces cumulative-current features and temporal lag construction as a compact way of representing ageing-relevant information. In the first study, this representation is examined with an MLP-based baseline; within the broader dissertation context, however, it serves mainly as the starting point of a larger development path that later extends toward recurrent architectures and deployment-oriented estimator design.
- A further methodological contribution is the robustness benchmark for embedded battery state estimation. Here, direct-measurement, hybrid, and data-driven estimator classes are compared under controlled disturbances such as bias, noise,

missing samples, timing irregularities, and initialization mismatch, so that estimator quality is assessed under deployment-relevant rather than purely nominal conditions.

- On the implementation side, the dissertation analyses how SOC and SOH estimators can be transferred to microcontrollers through pruning, quantization, and deployment-oriented STM32 benchmarking. This part of the work links model design to concrete trade-offs in memory demand, runtime latency, and practical firmware integration.
- The experimental basis of the dissertation is provided by the Battery Management System State-Estimation Test Platform (BMS-STEP), a custom hardware environment for repeatable embedded-AI studies within one common BMS architecture. This platform creates the technical basis for comparing interchangeable estimation methods under controlled and reproducible hardware conditions.
- Taken together, these elements form a cumulative design perspective for deployment-oriented battery intelligence that connects representation, robustness, embedded optimization, and hardware integration. The overall contribution therefore lies not only in the individual studies, but also in their integration into one coherent development and evaluation logic.

1.3 Specific Research Objectives

From this overall framing, the dissertation pursues five specific research objectives:

- To identify how neural-network methods can represent battery ageing with sufficient practical utility for stationary-storage applications and long-term condition monitoring.
- To assess under which disturbance conditions embedded battery state estimators preserve acceptable accuracy, convergence behaviour, and recovery capability.
- To evaluate how AI-based SOC and SOH estimators can be transferred to microcontrollers through resource-aware pruning, quantization, and model-to-device adaptation.
- To analyse under which architectural conditions pruning and quantization remain effective without disproportionately degrading predictive performance.

- To define how a hardware platform and validation chain should be structured in order to support repeatable, hardware-level evaluation of AI-based BMS methods under realistic embedded conditions.

1.4 Structure of the Dissertation and Development Path

The dissertation follows a cumulative structure in which study-specific methods, results, and local interpretation remain within the contribution chapters, while the common technical basis and the cross-study implications are developed in the surrounding chapters.[1, 2, 3, 4, 5, 6]

The progression of the work is intentionally staged. The ageing-estimation chapter first addresses how battery-relevant information can be represented in a neural estimator. The robustness benchmark then shifts the focus from nominal performance to disturbance-aware evaluation under realistic measurement corruption. The embedded-AI chapter extends this perspective toward microcontroller deployment, where compression, memory, and latency become decisive design variables. The subsequent compression chapter generalizes this question across architectures, while the hardware chapter and the BMS-STEP platform provide the common laboratory basis on which these deployment questions can be tested under realistic embedded conditions.

The shared laboratory hardware basis of the dissertation is the Battery Management System State-Estimation Test Platform (BMS-STEP), a custom 4s research BMS developed to compare and exchange embedded state-estimation methods on one reproducible system basis. The final synthesis chapter integrates the findings across all studies and derives the cumulative contribution of the work to deployment-oriented battery intelligence.

Tabelle 1.1: Cumulative structure of the dissertation and function of the core contribution chapters within the overall argument.

Order	Contribution chapter or basis	Primary dimension	Function within the overall argument
1	Experimental and hardware basis from the BMS-STEP platform and the associated reports	System context and validation basis	Establishes the shared measurement, firmware, and hardware environment in which the embedded-AI contributions of the dissertation are technically grounded.
2	Neural-network-based ageing estimation for stationary storage systems	Model design and ageing estimation	Establishes the representation-oriented starting point and shows how ageing-relevant information can be encoded through feature engineering and temporal input structure.
3	Robustness benchmarking of embedded battery state estimation	Robustness and deployment-facing performance	Defines the common deployment-oriented evaluation logic and shows why clean-condition error alone is insufficient for estimator selection.
4	Embedded artificial intelligence for battery management systems	Embedded deployment and optimization	Connects estimator design to practical microcontroller execution through pruning, quantization, latency, and memory measurements.
5	Architecture-dependent compression analysis	Compression sensitivity and co-design	Extends the embedded perspective toward broader model-hardware design rules across estimator families.

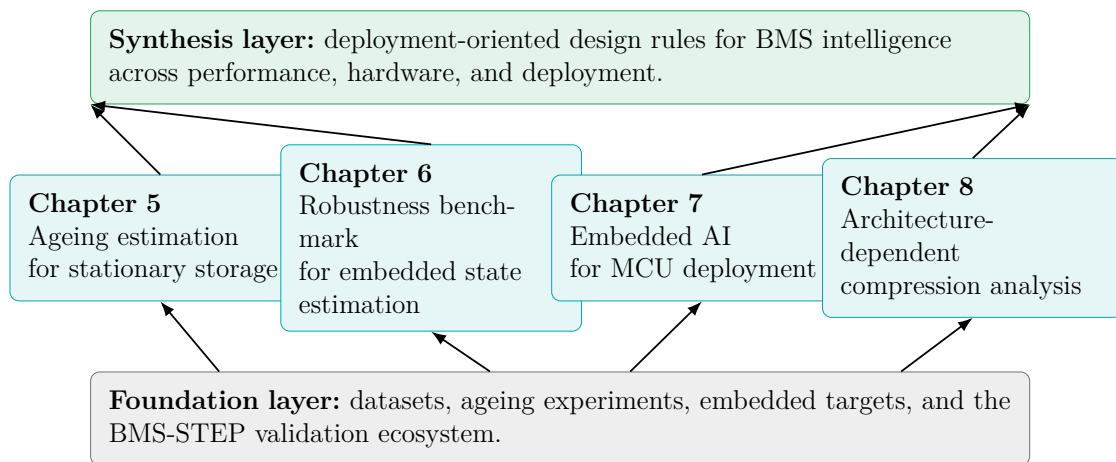


Abbildung 1.1: Roadmap of the overall work. The study chapters are embedded in a shared technical basis and are connected through a final synthesis toward deployment-oriented BMS intelligence.

Battery-Management Requirement Framework

Battery-management systems require estimation methods that satisfy several conditions simultaneously. In practical operation, a model must provide accurate and stable state information, remain robust under realistic measurement imperfections, fit within the available embedded resources, and be transferable into an executable BMS implementation. For this reason, the present work organizes the relevant battery-management requirements into three connected areas: performance requirements, hardware requirements, and deployment requirements. This framework provides the common basis on which the individual studies of the dissertation are interpreted. Each study emphasizes different requirement groups, but all of them contribute to the same overarching question of how neural-network methods can be developed for deployment-oriented battery management.

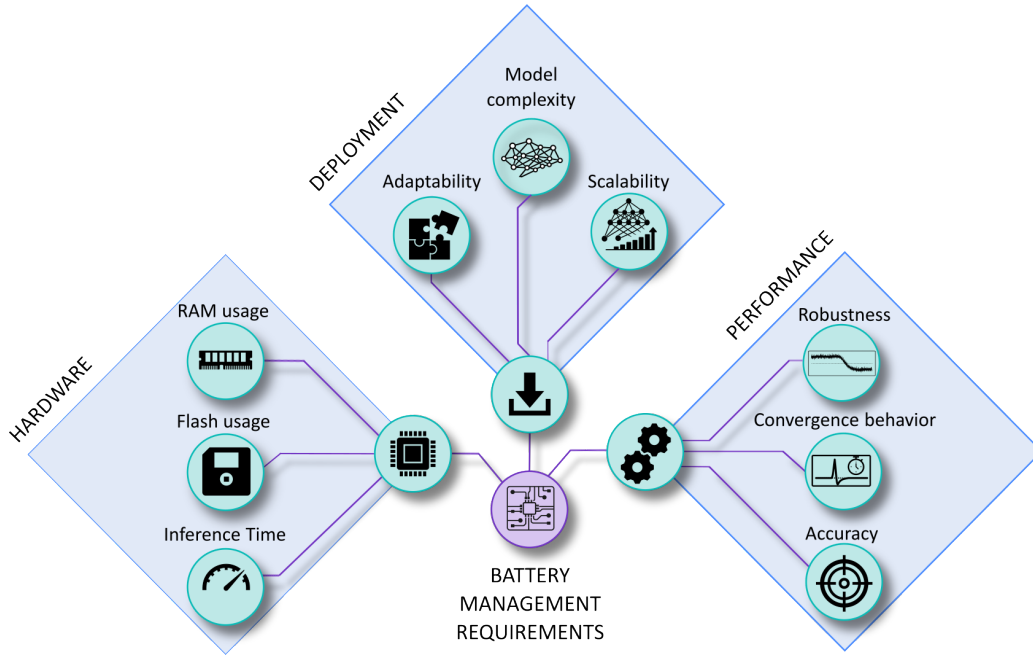


Abbildung 2.1: Battery-management requirement framework used throughout the dissertation. The framework groups deployment-oriented BMS requirements into performance, hardware, and deployment aspects and relates estimator quality to embedded feasibility and system realization.

2.1 Performance Requirements

Performance requirements describe the estimation quality that a BMS must obtain from an AI-based method during operation. In this dissertation, the central criteria are accuracy, robustness, and convergence behaviour. Accuracy is required because the estimator must provide reliable state information under nominal conditions. Robustness becomes essential once the input signals are affected by bias, noise, missing samples, or timing irregularities. Convergence behaviour determines whether the estimator can recover in a controlled manner after initialization mismatch, signal corruption, or temporary state inconsistency. These criteria are examined most directly in the robustness benchmark, but they also remain relevant when the results of the ageing-estimation and embedded-AI studies are interpreted in a broader deployment context.

2.2 Hardware Requirements

Hardware requirements arise because BMS estimators are executed on resource-limited embedded platforms rather than in unrestricted desktop environments. The most important quantities considered in this work are RAM usage, Flash usage, and inference time. These metrics determine whether a model can be executed on the target controller with sufficient timing margin and without exhausting the available memory. In addition, hardware-related conditions such as measurement quality, controller structure, and system integration influence which estimator concepts remain feasible in practice. These questions become particularly visible in the embedded-AI chapter, where memory footprint and latency are quantified directly, and in the hardware chapter, which provides the BMS-oriented environment in which such constraints can be assessed experimentally.

2.3 Deployment Requirements

Deployment requirements describe how readily a model can be transferred from research development into practical BMS use. In this dissertation, the most relevant aspects are model complexity, adaptability, and scalability. Model complexity influences both the effort of implementation and the feasibility of later compression and hardware transfer. Adaptability refers to the extent to which modelling concepts, feature constructions, and estimator structures can be reused or modified for related battery-state estimation tasks. Scalability concerns whether a method and its optimization strategy remain viable across different model classes and target platforms. This requirement area therefore connects feature design, architecture choice, pruning, quantization, and firmware transfer within one common perspective.

2.4 Coverage of the Framework in the Dissertation

- The ageing-estimation study addresses primarily the performance and deployment sides of the framework. Its focus lies on how ageing-relevant battery information can be represented in a neural estimator and how this representation can be translated into useful prediction quality for stationary-storage data.
- The robustness benchmark concentrates most strongly on performance requirements. It shows that estimator quality in battery management cannot be reduced

to nominal accuracy alone, but must also include robustness against disturbed measurements and convergence behaviour after inconsistencies.

- The embedded-AI study connects deployment requirements with hardware requirements. In this part of the dissertation, model transfer, pruning, quantization, RAM usage, Flash usage, and inference latency become central because they determine whether a method can actually be executed on a microcontroller target.
- The hardware chapter provides the experimental and system-level basis for this framework. It establishes a BMS-oriented platform on which different estimation methods can be integrated, exchanged, and tested under reproducible embedded conditions.
- The architecture-dependent compression study extends the deployment perspective toward more general design rules. It examines how strongly pruning and quantization depend on model class and how scalable such optimization strategies remain across different architectures.

Taken together, these studies cover complementary parts of one common battery-management requirement framework. The framework therefore functions as the conceptual link between the chapters and as the basis for the final synthesis of the dissertation.

State of the Art and Technical Background

3.1 Battery Management Systems

Battery management systems supervise cell voltages, current, temperature, balancing, protection paths, and communication with higher-level control systems. Within this functional structure, SOC and SOH are not isolated diagnostic variables, but central quantities for power limitation, charging control, lifetime-aware operation, and system-level energy management.

3.2 Neural Networks for Battery Ageing and State Estimation

Neural methods are attractive in battery applications because they can represent nonlinear signal relations, exploit temporal context, and combine measured and derived variables in a data-driven manner. At the same time, their practical value depends not only on predictive accuracy, but also on interpretability, training stability, robustness, and deployment effort.

3.3 Embedded AI in Battery Applications

Embedded AI in battery applications shifts the focus from model quality alone toward deployment quality. Relevant criteria therefore include not only prediction error, but also Flash demand, RAM demand, latency, numerical stability, reproducibility, and the effort required to transfer a model from high-level research code to microcontroller inference.

3.4 Compression and Model-Hardware Co-Design

Compression and model-hardware co-design become essential as soon as AI is treated as a practical BMS component. Pruning and quantization are therefore not secondary optimizations, but central design choices that influence memory footprint, runtime behaviour, numerical precision, and ultimately the feasibility of embedded deployment.

Experimental Basis and Hardware Ecosystem

- The experimental basis of this work combines laboratory battery datasets, embedded execution targets, and the custom BMS-STEP hardware platform for battery management.
- This combined setup makes it possible to study battery-related artificial intelligence not only as an offline modelling problem, but also as an embedded systems problem shaped by sensing, communication, firmware partitioning, and hardware constraints.
- The resulting experimental ecosystem forms the common basis for the ageing-estimation work, the robustness benchmark, and the embedded implementation studies.[1, 3, 2, 5, 6]

4.1 Data Sources and Ageing Campaigns

- The data basis of this work consists of two complementary domains. The first domain comprises ageing-oriented battery datasets used for the stationary-storage SOH study, whereas the second domain comprises laboratory datasets recorded in the context of the MG-FARM project and used for embedded SOC/SOH estimation and robustness analysis.[1, 3, 2]
- Across these datasets, the core measured quantities are voltage, current, temperature, and time. From these measured signals, additional reference quantities such as transferred charge, capacity, state of health, and cumulative charge state are derived for model development and evaluation.

- The laboratory campaigns were structured around controlled operating conditions and ageing states so that model behaviour could be analysed under reproducible charge and discharge trajectories instead of under loosely documented use cases.
- This separation between raw measurements and derived reference targets is important because it preserves a clear distinction between physically observed signals and offline evaluation quantities generated during preprocessing.
- The resulting data basis supports both long-term ageing analysis and deployment-oriented state-estimation studies on embedded hardware.

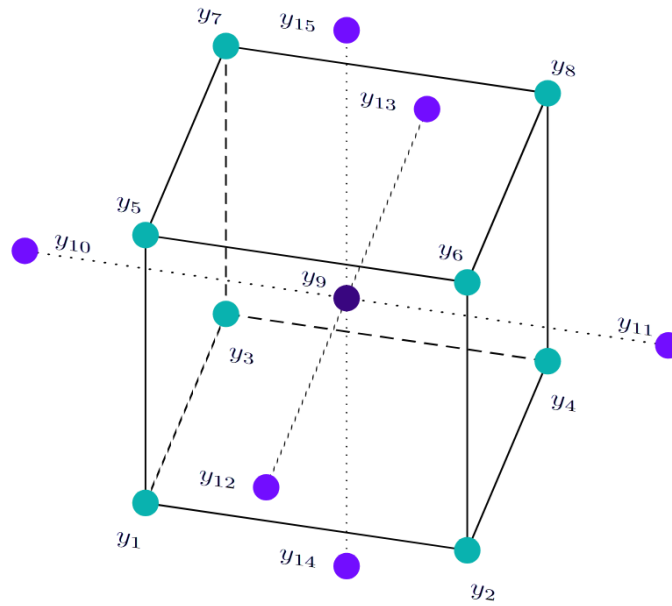


Abbildung 4.1: Design-of-experiments view of the laboratory ageing and operating-condition space used to structure the embedded battery test campaigns and the resulting datasets.

4.2 Embedded Targets and Execution Chain

- The embedded work relies on two complementary targets: the final BMS-STEP hardware and an STM32H755-based evaluation-board setup for rapid firmware iteration and controlled implementation tests.[5, 6, 3]
- The integrated BMS hardware is used whenever sensing, balancing, protection, communication, telemetry, and on-board state estimation must be validated as

one coupled system rather than as isolated software modules.

- The evaluation-board setup is used whenever estimator code, toolchain details, memory demand, and runtime behaviour have to be checked quickly before transfer to the full BMS-STEP platform.
- In both cases, the deployment path follows the same sequence described in the reports: state-estimation methods are first prototyped in Python or MATLAB, then rewritten as plain C modules, compiled into the STM32 firmware, and executed locally on the target hardware instead of off-board.
- During validation, the firmware records voltage, current, temperature, and relevant intermediate estimator variables. After each run, these traces are aligned with higher-fidelity offline reference calculations for SoC and SoH on a host PC. D7.11 explicitly states that no hardware-in-the-loop setup is used in this workflow.
- This split between evaluation-board work and integrated BMS operation is important for the dissertation because it separates pure implementation questions from questions of sensing, protection, and full system integration.

4.3 Custom BMS-STEP Platform

- Based on D7.4 and D7.11, the custom laboratory hardware developed in this work is referred to as the Battery Management System State-Estimation Test Platform, abbreviated as BMS-STEP.[5, 6]
- BMS-STEP is a physically realised 4s laboratory BMS designed to monitor and protect the cells, execute on-board SoC and SoH estimation, perform balancing, and provide the technical basis for controlled experiments with embedded battery intelligence.
- A central reason for the in-house design is adaptability. In contrast to a commercial off-the-shelf BMS, the BMS-STEP platform gives direct access to the sensing chain, firmware structure, interfaces, and estimator integration, so that state-estimation methods can be exchanged, modified, and benchmarked repeatedly on the same hardware basis.

- The board should therefore be understood less as a finished product and more as a configurable research platform that was built deliberately to support rapid iteration, customisation, and comparative testing.

4.3.1 System Objective and Laboratory Scope

- The present implementation is a compact and reproducible 4s laboratory BMS for battery ageing studies and embedded state-estimation experiments at TU Berlin EET.[5, 6]
- The platform is intended for ageing tests, cyclic and calendar experiments, and representative charging and discharging profiles rather than unrestricted field deployment in large battery racks.
- D7.11 makes clear that exact field loads are not reproduced one-to-one in the laboratory. Instead, they are approximated within a controlled setup that allows calibration, diagnostics, repeated validation, and structured dataset generation.
- This laboratory orientation is deliberate because it enables benchmark-style comparison of different state-estimation approaches under the same measurement chain, including classical model-based observers and modern data-driven or neural methods executed directly on the STM32 controller.
- The platform concept is explicitly geared toward experimentation: one hardware base should support multiple estimator variants without rebuilding the full BMS around each method.
- Validation campaigns reported in D7.11 include duty cycles derived from decentralized stationary-storage use cases with limited maintenance access. These tests were used to record voltage drift, current ripple, temperature gradients, and embedded estimator outputs under realistic operating conditions.

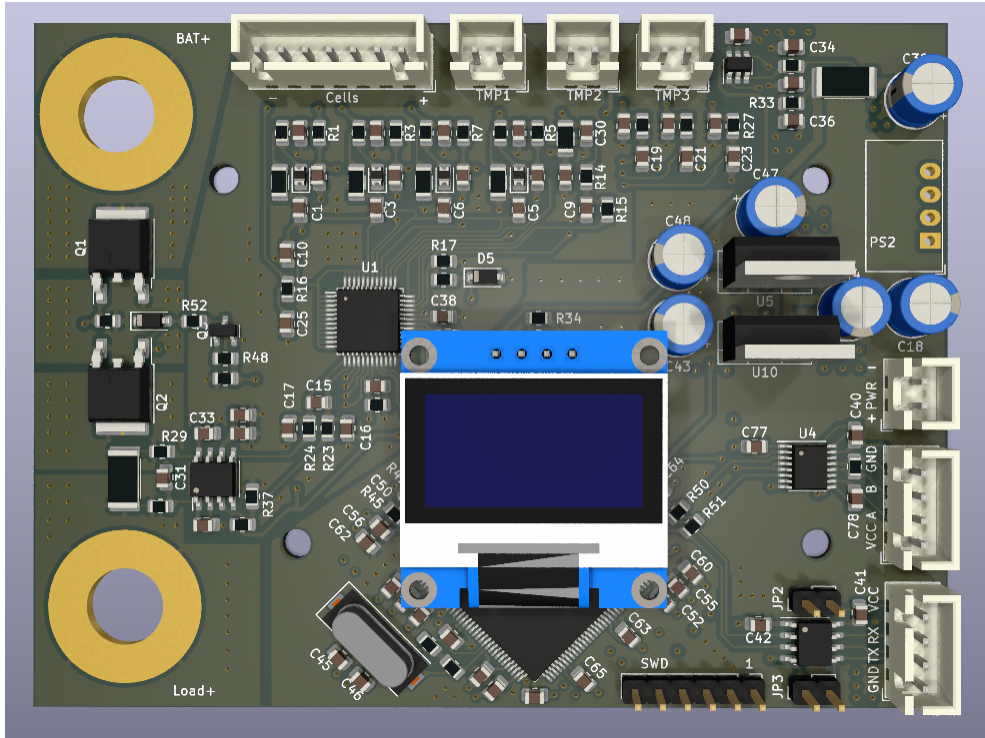


Abbildung 4.2: Board-level view of the custom BMS-STEP platform with integrated sensing, controller, communication, and auxiliary interfaces.

4.3.2 Hardware Architecture

- Both reports describe the BMS-STEP platform as three tightly coupled hardware domains: the TLE9012DQU-based sensing and balancing stage, the current-measurement and protection interface, and the controller/communication island around the dual-core STM32H755.[5, 6]
- The board layout keeps the high-current zone, shunt, and relay path physically separated from the low-voltage controller and communication domain. This separation matters not only for safety, but also for electromagnetic compatibility and stable measurement quality.
- The Infineon TLE9012DQU forms the measurement backbone of the pack. In the laboratory build it supervises four series-connected cells with buffered RC filtering and provides cell-voltage and temperature acquisition together with passive balancing capability.
- The active front-end includes balancing circuitry, while unused monitor inputs

are terminated according to the 12-to-4 cell adaptation of the reference design so that measurement integrity is preserved on the active channels.

- Temperature instrumentation is extended beyond the standard TLE inputs by an NTC harness that can connect additional probes at cell terminals, busbars, or ambient reference points. This provides a finer spatial thermal picture during ageing and state-estimation tests.
- The controller domain uses an STM32H755ZITx dual-core microcontroller. This design choice is central because it creates a clean architectural separation between conventional BMS functionality and the estimator layer that is investigated in the dissertation.

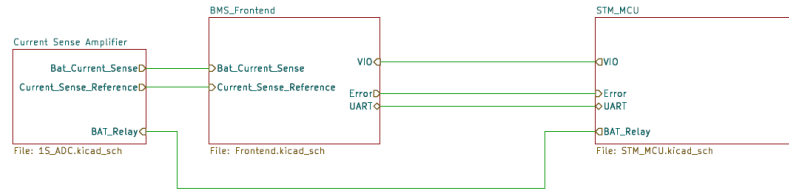


Abbildung 4.3: Reduced system-level block view of the BMS-STEP platform showing the interaction between current sensing, BMS front-end, and microcontroller domain.

4.3.3 Firmware and Execution Concept

- The firmware concept is intentionally centred on the dual-core STM32 architecture documented in D7.4 and D7.11.[5, 6]
- The Cortex-M4 core is used for conventional BMS functionality such as measurement handling, supervision, and general system management, whereas the Cortex-M7 core is reserved for the computationally heavier state-estimation routines.

- This split is important for the dissertation because it makes the estimator layer exchangeable. Different SoC and SoH methods can be integrated, tested, and replaced on the M7 side without redesigning the complete BMS firmware structure.
- The platform was built exactly for this purpose: one stable hardware and BMS basis should support repeated experimentation with different embedded-AI methods, different estimator classes, and different implementation variants.
- From the perspective of the dissertation, the central contribution of the firmware architecture is therefore not the low-level implementation detail itself, but the fact that it creates a reusable and customisable execution environment for embedded battery state estimation.

4.3.4 Experimental Relevance for Embedded AI

- The reports show clearly that the BMS-STEP platform is not only a monitoring board, but an experimental platform for comparing estimator classes under realistic embedded conditions.[5, 6]
- Its main value for the dissertation lies in controllability and openness: the same measurement hardware can be reused while state-estimation methods are changed, refined, compressed, or replaced.
- This makes it possible to compare classical model-based observers and data-driven or neural approaches on one common embedded basis instead of distributing them across unrelated hardware environments.
- The BMS-STEP platform therefore closes the gap between algorithm development and embedded validation. It was chosen precisely because it can be modified and extended in ways that would be difficult with a purchased closed BMS.
- At the same time, the 4s implementation remains deliberately compact. It serves as a reproducible laboratory baseline from which later transfer to larger stacks or a 16s/48 V variant can be reasoned without changing the overall control and estimator architecture.

Neural Network Architecture for Ageing Estimation in Stationary Storage Systems

5.1 Application Context

- This chapter addresses state-of-health estimation for lithium-ion cells intended for stationary storage systems in smart-grid applications.[1]
- In decentralized stationary systems and remote microgrid-like applications, this information is particularly relevant because long maintenance intervals and limited connectivity increase the need for reliable local ageing awareness inside the BMS.
- The starting point of the study is that stationary storage differs fundamentally from strongly dynamic mobile applications. The cells often operate around preferred operating windows, the current levels remain comparatively moderate, and the ageing state therefore has to be inferred from realistic operating data rather than from idealised full-cycle reference patterns alone.
- The paper positions this task as a data-driven estimation problem and deliberately uses a comparatively simple multilayer perceptron instead of a more complex recurrent architecture. The underlying claim is that the decisive performance gains do not come only from the network class itself, but from a suitable transformation of the measurement data into an ageing-informative feature space.
- The core novelty lies in the combination of cumulative-current feature engineering, lag-sequence generation, and systematic hyperparameter tuning. In this

way, the model is not limited to instantaneous values of current, voltage, and temperature, but also receives a compact representation of the electrochemical load history.

- In content, this chapter therefore follows the original paper closely: application context, ageing dataset, preprocessing, lag-sequence construction, MLP architecture, hyperparameter tuning, and the resulting prediction accuracy are transferred almost one-to-one, but reformulated into dissertation-style bulletpoints.

5.2 Raw Cell Dataset, Parameters, and Measured Quantities

- The underlying dataset originates from cyclic ageing tests on Li-ion INR18650 NMC cells carried out at Technische Universität Berlin.[1]
- The investigated cells have a nominal capacity of 2,6 A h, a nominal voltage of 3,7 V, a maximum charging voltage of 4,2 V, a discharge cut-off voltage of 2,75 V, and a maximum continuous discharge current of 5C.
- In total, 14 fresh cells are analysed. They are grouped into seven ageing scenarios with two cells per scenario, and all cells start from an initial SOH of 100 %.
- The scenarios vary four operating parameters: the mean state of charge, the depth of discharge, the discharge C-rate, and the operating temperature. These parameters are selected because they reflect relevant operating regimes for stationary storage systems in smart grids.
- Charging is performed with a constant-current constant-voltage procedure, whereas discharging is carried out with constant current. After a defined number of ageing cycles, capacity tests are inserted to determine the degradation state of the cells.
- The directly measured quantities are current, voltage, temperature, and time. Because the original sampling intervals depend on the respective process step, all measurements are resampled to 1 s before the later feature engineering is applied.

- One scenario with 20% mean state of charge, 30% depth of discharge, 1C discharge rate, and 35°C contains premature test termination and irregular measurements for cells 3 and 4. The paper deliberately keeps these data in the training pool in order to examine whether the model can still learn meaningful ageing relations from imperfect measurements.

Tabelle 5.1: Ageing scenarios of the stationary-storage dataset transferred from the MDPI paper.

Scenario	Cell No.	SOC _{mean}	DOD	C-Rate _{dis}	T (°C)
1	1 and 2	50%	100%	1C	35
2	3 and 4	20%	30%	1C	35
3	5 and 6	50%	30%	1C	35
4	7 and 8	80%	30%	1C	35
5	9 and 10	50%	100%	2C	35
6	11 and 12	50%	100%	1C	45
7	13 and 14	50%	100%	2C	45

5.3 Preprocessing Method and Feature Engineering

5.3.1 Cumulative Current

- The preprocessing pipeline is built around the argument that instantaneous current alone is not sufficient as an explanatory variable for ageing estimation.[1]
- The paper therefore introduces cumulative current as the central derived feature. This quantity integrates current over time and thereby reflects both the present loading level and the throughput-related stress that accumulates over the lifetime of the cell.
- The original argument is explicitly energy-related: the model should not only see whether a cell is currently charging or discharging, but should also receive information about how much electrochemical throughput has already occurred within the observed history window.
- The work further argues that charging and discharging should not simply be collapsed into one absolute current magnitude. Because charging and discharging

can lead to different voltage gradients and different effective ageing signatures, the representation is designed to retain directional information as far as possible.

- In this formulation, cumulative current becomes the feature that allows a relatively simple feedforward network to still access ageing-relevant temporal context without explicitly using a recurrent structure.
- Voltage and temperature are retained as further direct inputs, so the final feature set combines operating-state information with a compact representation of the prior load exposure.

5.3.2 State-of-Health Reference

- The SOH target is obtained from periodic capacity tests.[1]
- For each cell, the measured capacity is related to the nominal capacity so that the state of health is defined as the ratio between actual and nominal capacity.
- The first capacity test serves as the nominal reference point, whereas the following tests describe the degradation progression over time.
- Because current, voltage, and temperature are available continuously but the capacity tests exist only at discrete points, the SOH progression is interpolated onto the timeline of the directly recorded measurements.
- The paper represents this SOH progression both over equivalent full cycles and over absolute time in days. The temporal representation is especially relevant for the neural network because it makes the long-term ageing trend visible in the same time domain from which the lag sequences are extracted.
- The resulting SOH trajectories already reveal that temperature has a pronounced impact on degradation, whereas the interactions between the remaining operating parameters are less obvious from direct visual inspection alone.

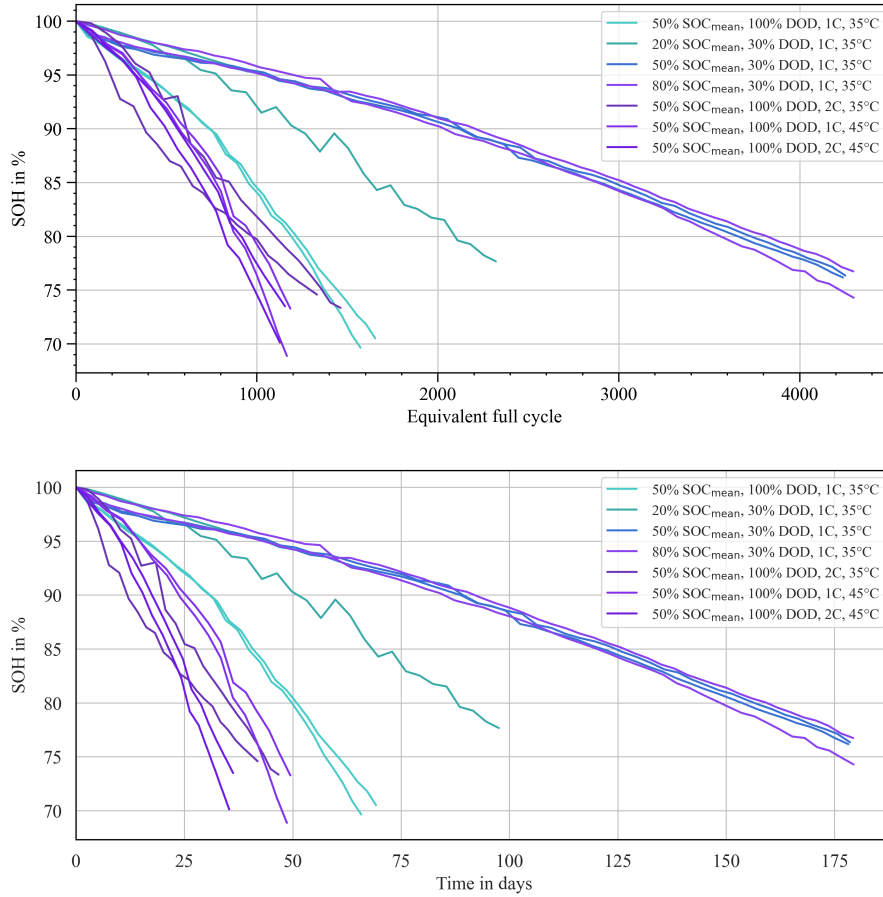


Abbildung 5.1: SOH progression of the 14 cells over equivalent full cycles and over absolute time, transferred from the MDPI paper.

5.3.3 Correlation Analysis

- To better understand which measured quantities should be retained for learning, the original paper supplements the SOH trajectories with a correlation analysis.[1]
- The resulting heatmap indicates that temperature and cumulative current, both in positive and negative direction, show the clearest direct relation to SOH.
- In contrast, the time derivative of the voltage does not show a comparably direct influence in the linear correlation view, although the paper explicitly notes that this does not exclude non-linear relevance for neural-network-based inference.
- The correlation analysis is therefore treated only as a starting point for feature selection and not as a replacement for the later model-based evaluation. Still, it

provides a concrete justification for keeping temperature and cumulative-current descriptors in the final feature set.

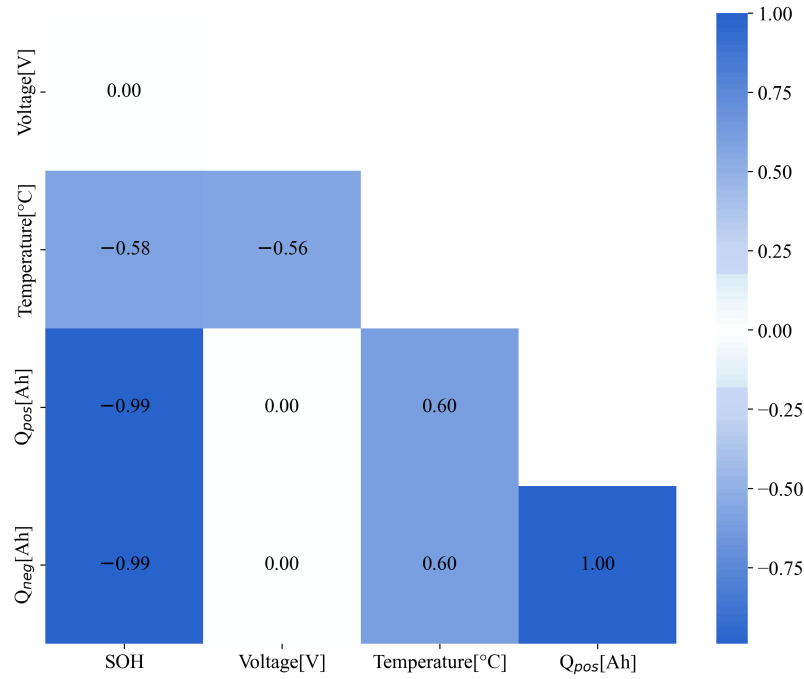


Abbildung 5.2: Correlation heatmap used in the paper to identify direct linear relations between SOH and candidate input variables.

5.3.4 Formation of the Lag Sequence

- Each learning sample is constructed as a lag sequence of voltage, temperature, and cumulative current, with the SOH at the current point in time used as the corresponding label.[1]
- This transforms the continuous measurement stream into structured sliding-window samples that can be processed by a feedforward network without passing the complete life history of an individual cell directly into the model.
- The initial parameterisation uses a time resolution of 30 min and 12 lag steps, which corresponds to a history length of 6 h.
- The paper argues that the time resolution has to match the actual charging and discharging behaviour. If the spacing is too coarse, relevant dynamics disappear. If it is too fine, the model sees an unnecessarily long and noisy representation that can make training less effective.

- The original text also links this design to the sampling theorem: the spacing between data points has to remain compatible with the temporal structure of the charging and discharging cycles so that the network can still recognise the relevant periodic behaviour.
- Because the raw measurements are resampled before sequence generation, the preprocessing step also acts like a low-pass filter and reduces the effect of high-frequency noise components.
- A further benefit of the lag-sequence approach is that the training samples are no longer tied to one specific cell trajectory. Instead of memorising individual cells, the model is trained on many local windows drawn from the full dataset.

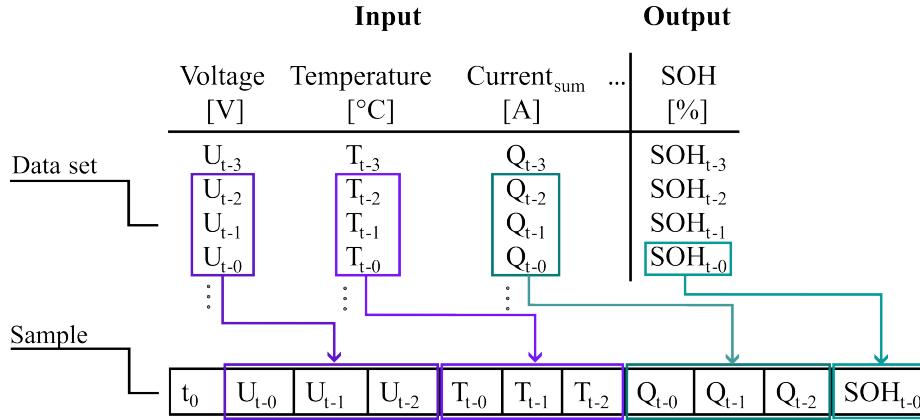


Abbildung 5.3: Lag-sequence construction used to transform raw cell measurements into supervised MLP samples.

5.3.5 Training, Validation, and Test Splits

- The paper separates the available data into training, validation, and test subsets.[1]
- During training, 20% of the available training sequences are reserved for validation so that the fitting process can be monitored for overfitting and generalisation behaviour.
- At the same time, several complete cells are withheld from the training process for the final performance assessment. The paper explicitly states that four out of fourteen cells are retained for this independent accuracy evaluation.

- This choice is important because the method is intended to generalise to previously unseen cells and not only to previously unseen windows extracted from known cells.
- The resulting process chain therefore mirrors the actual intended use case: build the model on one group of cells, monitor generalisation on validation data, and finally evaluate the trained estimator on cells that were not part of the optimisation loop.

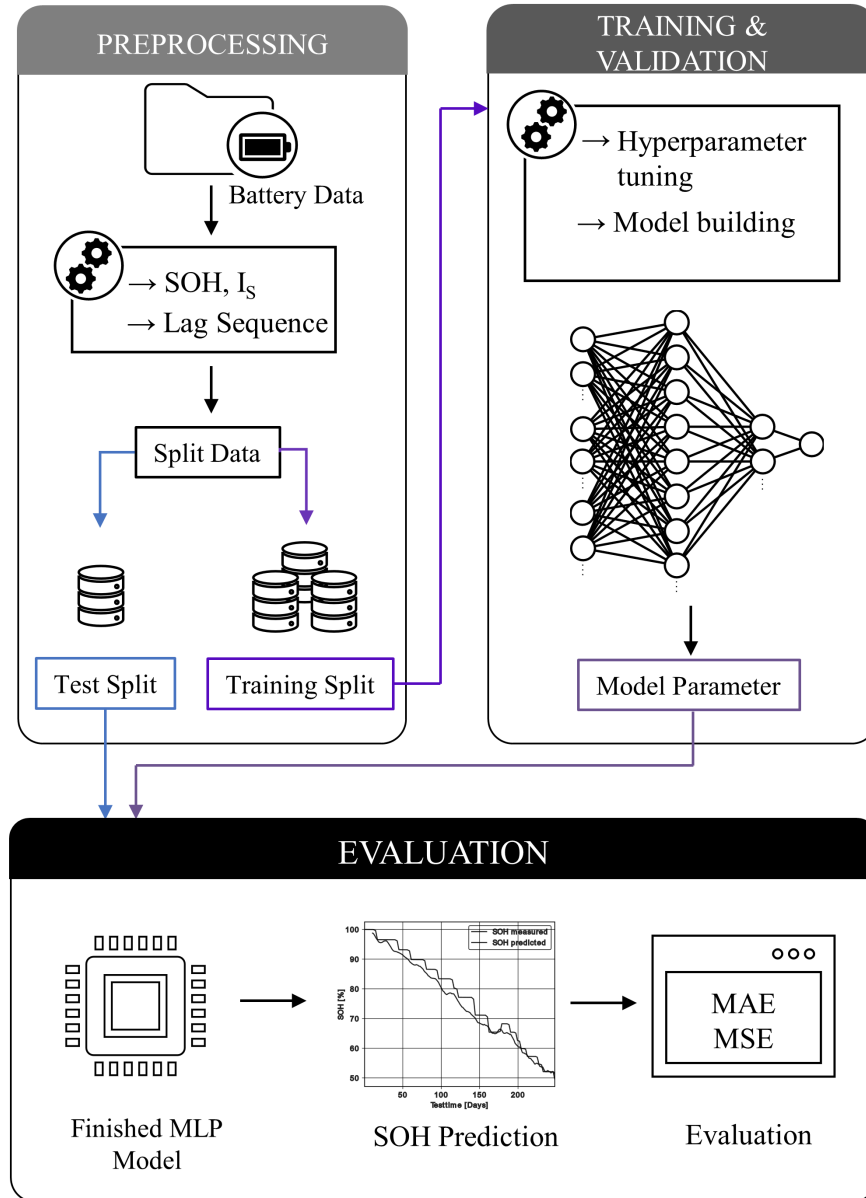


Abbildung 5.4: Processing and evaluation workflow of the MLP-based SOH estimator, transferred from the MDPI paper.

5.3.6 Data Scaling

- All input features are scaled to the interval between 0 and 1 before training.[1]
- This step is necessary because the numerical ranges of the features differ strongly. Voltage remains in a narrow interval between 2,75 V and 4,2 V, whereas cumulative current can rise to several thousand ampere-hours across the full

ageing experiment.

- Uniform scaling improves numerical stability, supports convergence during optimisation, and avoids a situation in which large-magnitude inputs dominate the network weights while smaller but still relevant signals are effectively ignored.

5.4 Multilayer Perceptron Model Structure

- The selected model type is a multilayer perceptron with fully connected layers.[1]
- The paper explicitly argues against a convolutional interpretation because the data do not exhibit a spatial image-like structure. It also refrains from directly using recurrent architectures such as LSTMs, because the temporal dependence is already encoded through the lag-sequence construction and because the recurrent alternatives are more resource-intensive and more difficult to tune.
- The MLP therefore consists of an input layer, several dense hidden layers with ReLU activation, and a single linear output node for the SOH estimate.
- The original starting point is intentionally small and uses one hidden layer with eight nodes. The final network depth and width are then determined in the hyperparameter search.
- In the tuned configuration, the model grows into a deep dense architecture with nine hidden/output stages and a final linear regression output. The resulting structure remains conceptually simple from an architectural point of view, but it is substantially larger than the initial baseline.

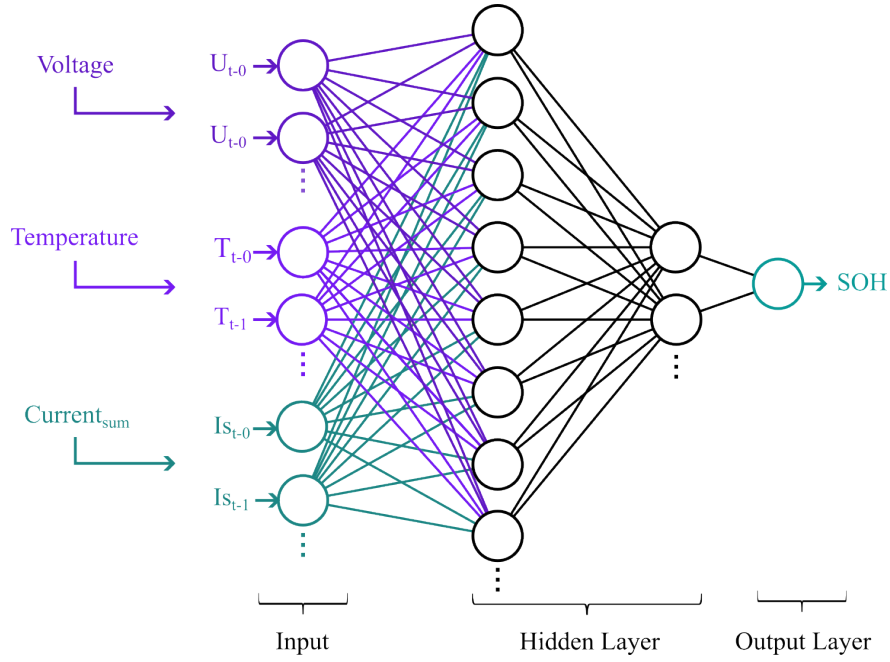


Abbildung 5.5: MLP architecture used in the MDPI paper for SOH estimation.

Tabelle 5.2: Tuned MLP configuration transferred from the MDPI paper.

Layer	Nodes	Activation Function
0	128	ReLU
1	256	ReLU
2	256	ReLU
3	256	ReLU
4	128	ReLU
5	512	ReLU
6	128	ReLU
7	256	ReLU
8	256	ReLU
9	1	Linear

5.5 Evaluation and Hyperparameter Tuning

- Model quality is evaluated by means of mean squared error and mean absolute error.[1]

- Mean squared error is used as the central optimisation-oriented metric during training and validation, whereas both MSE and MAE are used for the final interpretation on the held-out test cells.
- The hyperparameter search covers the number of lag steps, the temporal resolution of the lag sequence, network depth, number of nodes, batch size, kernel regularisation, optimizer type, and activation function.
- Hyperband is used as the main search strategy. The reason given in the original paper is that Hyperband distributes training resources adaptively, evaluates many candidate settings quickly, and allocates more epochs only to the most promising configurations.
- The study reports that ADAM outperforms RMSProp, Adadelta, and Adagrad for this task. Larger batch sizes reduce overfitting tendencies, whereas explicit kernel regularisation does not lead to a meaningful improvement.
- The optimizer analysis is complemented by an activation-function comparison. ReLU yields the lowest loss in the tested setup, while sigmoid, tanh, and softmax perform worse.
- The original paper also states that the models are trained on approximately 15,000 sequences with varying batch-size and epoch combinations and that larger batches converge to a comparable minimum while reducing overfitting tendencies.

5.6 Results and Performance Evaluation

- The evaluation shows that the two most influential sequence parameters are the number of lag steps and the temporal resolution.[1]
- For the detailed analysis, the paper compares four time resolutions: 1 min, 10 min, 30 min, and 60 min. In parallel, the number of lag steps is varied between 8, 12, 16, and 20.
- Across the exemplary cells, the most favourable time-resolution range lies between 10 min and 30 min. This matches the characteristic charge and discharge durations of the investigated scenarios, which typically fall in the range of approximately 20 min to 1 h.

- If the time resolution is too large, relevant behaviour is averaged out and important information is lost. If it is too small, the training process becomes less effective and the longer effective histories do not translate automatically into better prediction quality.
- The original MAE matrix shows the smallest errors for 16 lag steps at a time resolution of 10 min. This corresponds to a history length of 160 min.
- Extending the sequence length beyond 20 steps is reported as disadvantageous because the resulting history becomes excessively long and does not improve prediction quality further.

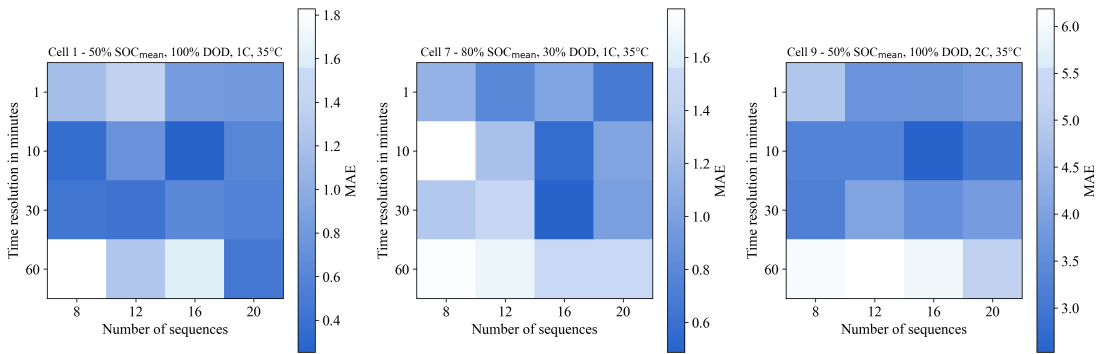


Abbildung 5.6: MAE matrix from the MDPI paper showing the interaction between sequence length and temporal resolution.

- The trajectory comparison between predicted and measured SOH shows that cells with typical ageing behaviour are reproduced well, whereas a cell with a pronounced U-shaped SOH progression leads to a substantially larger prediction error.
- The original paper discusses cells 7 and 1 as representative examples of typical ageing behaviour and cell 9 as the deliberately selected atypical case.
- For the favourable individual cells, the reported best errors are MSE values of 0.410 and 0.118 and MAE values of 0.487 and 0.253.
- Across the evaluated cases, the paper reports an average MAE of 1.10 and an average MSE of 2.95.

- The prediction plots additionally show that a post-rolling filter is applied to reduce visual noise and to make the comparison between measured and estimated SOH easier to interpret.

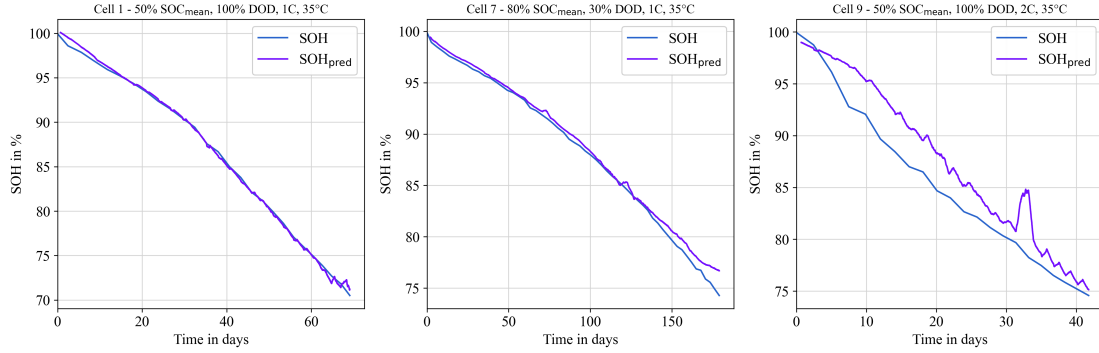


Abbildung 5.7: Comparison of predicted and measured SOH trajectories for three representative test cells.

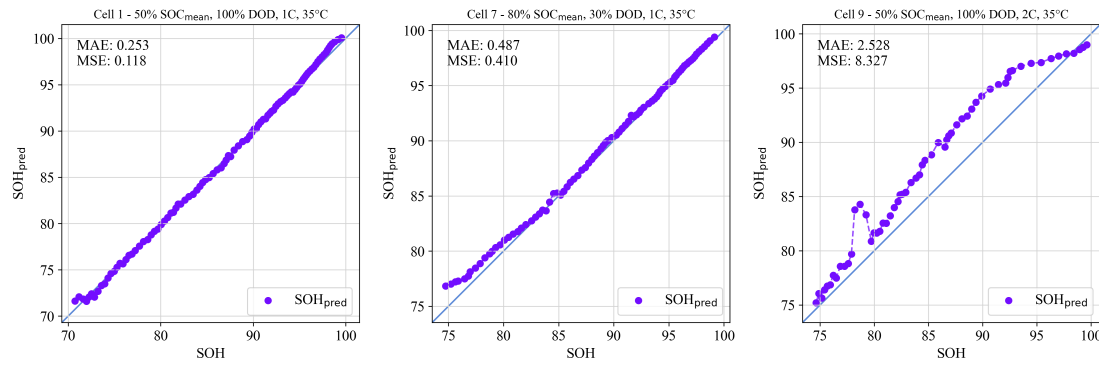


Abbildung 5.8: Predicted versus measured SOH values for the representative test cells used in the MDPI paper.

- The paper also investigates the effect of removing the problematic cells 3 and 4 from the training basis. In that variant, the reported aggregate error improves to an MAE of 1.07 and an MSE of 2.52.
- At the same time, the behaviour on the atypical cell 9 becomes more unstable, which the original paper interprets as a sign that cleaner data alone do not replace broader data diversity.
- The result section therefore already makes a broader methodological point: the exact optimum for sequence length and time resolution depends strongly on the

available dataset, and direct comparison with other literature remains difficult if the underlying cell data differ substantially.

5.7 Interpretation of the Ageing-Estimation Results

- The chapter shows that a meaningful SOH estimate can be obtained with a comparatively simple feedforward model when the preprocessing and feature engineering are aligned with the underlying ageing problem.[1]
- One of the central insights is that the input representation is at least as important as the model class itself. Cumulative current, lag-sequence construction, and interpolation of the SOH reference are decisive parts of the overall method.
- The results simultaneously show the dependence of neural SOH estimation on the available dataset. Atypical ageing trajectories, interrupted tests, and limited data diversity remain the dominant constraints on generalisation.
- Even under these limitations, the approach reaches a level of accuracy that is competitive with more complex alternatives reported in the paper. This is precisely why the study is relevant for the dissertation: it establishes a first data-driven estimator that remains simple enough to connect later to questions of robustness, compression, and embedded deployment.
- The original discussion also stresses that the method is not restricted to one narrowly defined technology, because it relies only on directly measured quantities such as voltage, temperature, and current-derived throughput features. In principle, the concept can therefore be transferred to other cells and chemistries if sufficiently rich data are available.
- As future work, the paper explicitly points to broader datasets and to comparisons with CNN- and LSTM-based alternatives in order to test whether the proposed lag-sequence concept remains advantageous across different model classes.

Robustness Benchmarking of Embedded Battery State Estimation

6.1 Application Context and Objective

- This chapter investigates deployment-facing battery-state estimation under realistic Battery Management System conditions rather than under clean laboratory input signals alone.[2]
- The central objective is a measurement-only robustness benchmark in which estimator performance is isolated from broader system-level effects such as hardware-resource optimization or implementation-effort comparison.
- Practical deployment quality is not defined by clean-condition accuracy alone, because real BMS operation is exposed to sensor bias, additive noise, missing samples, irregular timing, restart-related initialization mismatch, and transient signal spikes caused by the measurement chain or by communication faults.
- In this benchmark, estimator quality is therefore defined as the joint behavior under nominal baseline conditions, under initialization mismatch, and under disturbed measurements or signal-integrity faults.
- The chapter compares four embedded-ready estimator classes on the same LFP full-cell trajectory and under the same disturbance protocol: a direct-measurement model (DM), a hybrid direct-measurement model with learned SOH support (HDM), a hybrid equivalent-circuit model with voltage feedback (HECM), and a data-driven recurrent model with GRU-based SOC estimation and LSTM-based online SOH context (DD).

- Under clean conditions, HDM provides the lowest baseline error with an MAE of 0.0024, followed by HECM with 0.0090, DD with 0.0100, and DM with 0.0311. Under disturbed measurements, however, the ranking changes substantially, which is the core reason for the benchmark.
- The benchmark therefore shifts the evaluation logic away from single clean-condition error values and toward a more realistic view in which sensing imperfections, missing updates, and recovery after corrupted inputs become part of the core assessment.

6.2 State of the Art and Benchmark Gap

- The relevant literature usually distinguishes between direct-measurement or integration-based estimators, model-based estimators built around equivalent-circuit or electrochemical structure, hybrid approaches, and purely data-driven sequence models.[2]
- Direct-measurement approaches are attractive because they are simple, computationally inexpensive, and easy to deploy, but they are structurally exposed to current bias, missing data, and initialization errors because they do not contain an internal correction mechanism.
- Equivalent-circuit and observer-based estimators offer physical interpretability and voltage-based correction capability, but their practical performance depends strongly on parameter quality, observability, and measurement quality.
- Hybrid approaches combine physical state propagation with learned adaptation of capacity, parameters, or auxiliary states and are therefore attractive for joint SOC/SOH estimation, but they do not automatically remove disturbance sensitivity.
- Data-driven recurrent estimators can exploit nonlinear relations between voltage, current, temperature, and derived online variables, and recent work has also shown their embedded feasibility through pruning, quantization, and TinyML-style deployment.
- The main benchmark gap is that much of the literature still emphasizes clean-data accuracy, average error metrics, or computational efficiency, while comparatively few studies compare fundamentally different estimator classes under

the same disturbance protocol and under matched embedded-ready execution conditions.

6.3 Estimator Classes Under Test

- The benchmark compares four embedded-ready estimator classes under matched online execution conditions.[2]
- DM represents the canonical Coulomb-counting baseline with fixed nominal capacity and explicit full-charge reset logic. It is the low-complexity reference for integration-driven SOC estimation.
- HDM preserves the same Coulomb-counting core but replaces the fixed capacity assumption with an online effective capacity obtained from a shared SOH estimator. In structural terms, it differs from DM only in the denominator of the SOC mapping and therefore represents a lightweight hybrid estimator.
- HECM represents the physics-based observer class. It is implemented as a second-order RC model with the state vector $\mathbf{x}_k = [\widehat{\text{SOC}}_k, U_{1,k}, U_{2,k}]^\top$, current-driven state propagation, voltage-residual correction, and SOH-dependent parameter scheduling.
- DD represents the compact recurrent neural estimator class. It uses a stateful GRU-based SOC model with the feature vector $\{U, I, T, \text{SOH}, Q_c, dU/dt, dI/dt, \Delta t\}$, a single-layer GRU with hidden size 96, an MLP head with one hidden layer of size 96, and a sigmoid output layer.
- All SOH-aware variants share the same LSTM-based online SOH estimator, so that the comparison focuses on how the estimator classes consume the shared SOH information rather than on unrelated SOH-model differences.
- The shared SOH estimator operates on hourly aggregated online features derived from $\{U, I, T, \text{EFC}, Q_c\}$ using mean, standard deviation, minimum, and maximum, which yields a 20-dimensional hourly feature vector. Architecturally, it consists of a feature embedding block of size 128, a two-layer unidirectional LSTM with hidden size 160, and a prediction head with hidden size 160.
- During benchmark execution, the SOH estimate is updated at hourly cadence and then held piecewise constant between update points. In HDM it rescales the

effective capacity, in HECM it affects the capacity and lookup-based parameterization, and in DD it is provided as an explicit causal input feature.

- The direct-measurement variants use the same online cumulative charge state Q_c , rebuilt causally from the measured current and the sampling interval. HDM replaces the fixed nominal capacity with the effective capacity relation $C_{\text{eff}} = C_{\text{nom}} \cdot \widehat{\text{SOH}}$.

6.4 Embedded-Oriented Model Preparation

- Although the central focus of the chapter is estimator performance and not hardware optimization, all implementations are selected and prepared under a common embedded feasibility requirement corresponding to an STM32H755ZI-class platform with 1 MB flash and 1 MB RAM.[2]
- DM and HECM are already compact by construction because they rely on scalar charge-state updates and lookup-based observer logic. For the neural components, deployment-prepared variants are used.
- These compact neural variants correspond to structured width reduction, short post-pruning finetuning, and INT8 weight quantization, yielding hidden size 67 for the SOC-GRU and hidden size 112 for the SOH-LSTM in the deployment-prepared versions.
- The component-level footprint estimates in the paper are 4.2 KiB flash and 16 B for the DM core, 27.9 KiB flash and 268 B for the SOC-GRU core, 408.3 KiB flash and 1792 B for the SOH-LSTM core, and 442.5 KiB flash and 52 B for the HECM core.
- Aggregated to estimator level, the reported persistent footprints are 4.2 KiB flash and 16 B RAM for DM, 412.5 KiB and 1808 B for HDM, 850.8 KiB and 1844 B for HECM, and 436.2 KiB and 2060 B for DD.
- These values matter because the benchmark does not compare unconstrained large neural models against inherently compact classical estimators. All four variants remain within the same reference deployment budget.

6.5 Measurement-Only Benchmark Principle

- The benchmark follows a strict measurement-only principle.[2]
- Only quantities that a real BMS measures directly or reconstructs online from measured data are manipulated, namely current, voltage, temperature, sample availability, and sampling time.
- Artificial hidden-state corruption or free synthetic parameter mismatch are deliberately excluded, because the benchmark is intended to remain physically interpretable and reproducible from the perspective of practical BMS sensing.
- Under this design, all estimators process the same disturbed input stream online and rebuild their auxiliary features causally from the corrupted measurements, including cumulative charge, derivative channels, and SOH-related context where applicable.
- The benchmark should therefore be read as a controlled cross-class estimator study and not as a full production-BMS validation campaign. Its role is to isolate estimator-side failure modes under matched data, matched disturbance definitions, matched online execution rules, and matched embedded-feasibility constraints.
- This ensures that the observed differences can be attributed to estimator structure and disturbance propagation instead of to mismatched preprocessing or access to unavailable information.

6.6 Disturbance Taxonomy and Scenario Design

- The disturbance space is grouped into input disturbances, initialization errors, and signal-integrity faults.[2]
- The implemented scenarios are ADC quantization, current bias, additive current noise, voltage noise, temperature noise, initial SOC mismatch, missing samples, irregular sampling, burst dropout, and voltage spikes.
- ADC quantization is implemented with step sizes of 0,01 A for current, 0,005 V for voltage, and 0,5 °C for temperature.

- Current bias is implemented as a constant offset of 0,05 A in the compact matrix and additionally as a dedicated sweep of 0.5%, 1.5%, and 3.0% of $|I|_{\max}$.
- Current noise is modelled as Gaussian noise with $\sigma_I = 0,10$ A in the compact matrix and is extended to a local-detail sweep up to $\sigma_I = 0,20$ A.
- Voltage noise is implemented as additive Gaussian noise with $\sigma_U = 0,01$ V, and temperature noise as additive Gaussian noise with $\sigma_T = 1,0$ °C.
- The initialization scenario applies an additive initial mismatch of -0.10 SOC. For DD this is realized as a perturbation of the online cumulative-charge feature Q_c rather than as an explicit latent-state reset.
- Missing samples are implemented by freezing every 50th sample on the nominal 1 s grid. Irregular sampling uses uniform timing jitter of $\pm 0,1$ s around the nominal 1 s grid.
- Burst dropout is implemented as one central frozen gap of 3600 s, and the voltage-spike scenario injects one single-sample spike of $\pm 0,20$ V every 1000 samples, corresponding to every 1000 s on the nominal time base.
- These scenarios are selected because they represent sensor limitations, adverse measurement corruption, and deliberate signal-integrity stress cases that remain meaningful for embedded BMS deployment.
- Current bias primarily exposes long-term integration drift and sensor miscalibration. Voltage noise primarily stresses the correction pathway of physics-based and hybrid estimators. Missing samples, burst dropout, and irregular sampling expose the behavior under frozen updates, delayed information, and timing uncertainty.
- The disturbance taxonomy is therefore not only a list of corruptions. It is a structured way of exposing different failure mechanisms such as drift, loss of correction, delayed resynchronization, transient instability, or persistent post-disturbance error.

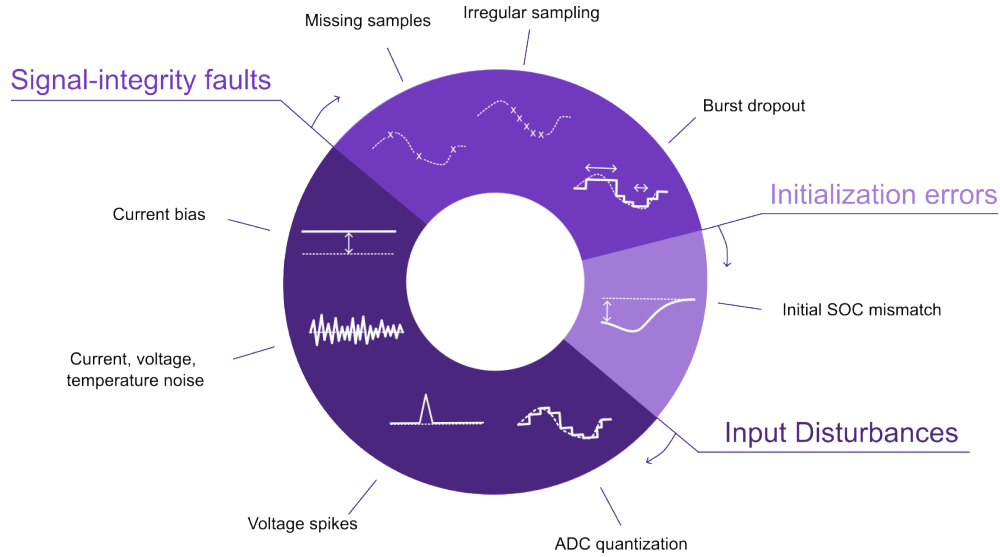


Abbildung 6.1: Taxonomy of the disturbance scenarios used in the robustness benchmark, grouped into input disturbances, initialization errors, and signal-integrity faults.

6.7 Metrics and Evaluation Logic

- The benchmark evaluates three performance dimensions: nominal accuracy, convergence behavior after state inconsistency, and robustness under sustained or transient measurement corruption.[2]
- Global disturbed-operation performance is quantified through MAE, RMSE, bias, maximum error, and the 95th percentile absolute error. These metrics summarize average accuracy, systematic deviation, and tail behavior across the complete trajectory.
- Local disturbance performance is additionally analysed through scenario-specific quantities such as output variance, excess rolling MAE, recovery time, jump behavior, and aligned post-disturbance error, because several scenarios do not primarily manifest as a collapse of full-run MAE.
- Whenever a disturbance mask exists, the analysis separates pre-disturbance, disturbed, and post-disturbance behavior in order to quantify disturbance-window error growth, recovery time, and residual post-disturbance error.
- In the current benchmark phase, ADC quantization, current bias, voltage noise, temperature noise, missing samples, and irregular sampling are interpreted main-

ly through global metrics, whereas burst dropout, voltage spikes, current-noise detail, and initialization mismatch additionally require local evidence.

- Recovery after initialization errors is interpreted through two baseline-relative operating bands rather than through one universal fixed threshold. The strict band is defined as $\max(\text{P95}_{\text{base}}, 1.5 \cdot \text{MAE}_{\text{base}})$.
- The fair band is defined as $\max(2 \cdot \text{P95}_{\text{base}}, 3 \cdot \text{MAE}_{\text{base}}, 0.02)$ and acts as a more deployment-tolerant criterion for estimators that re-enter a practically acceptable operating range without immediately returning to their strict near-baseline regime.
- Both recovery bands require sustained re-entry over a fixed horizon so that the reported recovery times reflect stable resynchronization rather than an accidental short crossing of the threshold.
- This multi-scale metric design evaluates performance as a structured combination of nominal accuracy, convergence behavior, global degradation, local disturbance response, and long-term drift.

6.8 Dataset Basis and Online Execution Setup

- The benchmark is based on an LFP 18650 laboratory dataset recorded in the context of the MG-FARM project, which provides long measured full-cell trajectories together with sufficient structure for reproducible disturbance injection and reference-target generation.[2]
- In the current benchmark phase, one representative full-cell trajectory is used in order to isolate estimator behavior under controlled disturbance scenarios before extending the study to additional cells and cross-cell validation.
- The available raw channels are current, voltage, temperature, and time. Offline reference targets for SOC and SOH are derived from charge integration, capacity-test information, and interpolated degradation progression.
- In the preprocessing of the paper, the signed transferred charge is computed as $Q_k = I_k \Delta t_k / 3600$, the absolute transferred charge as $|Q_k|$, and the reference capacity as $C_{\text{ref}} = \sum_k |Q_k|$ during dedicated capacity-test intervals.

- The offline SOH target is then defined as $\text{SOH}_{\text{ref}} = C_{\text{ref}}/C_{\text{ref},0}$, where $C_{\text{ref},0}$ denotes the first measured reference capacity of the trajectory. The reference SOC target corresponds to the offline Coulomb-counting target $\text{SOC}_{\text{ref}} = 1 + Q_c/C_{\text{ref},0}$.
- All compared estimators are executed under online conditions. Disturbed inputs are processed sample by sample, and all required auxiliary quantities are rebuilt directly from the disturbed signals rather than copied from precomputed dataset columns.
- In freeze and dropout scenarios, the disturbance propagation is handled consistently by freezing or distorting the derived online features together with the corresponding measured inputs, which is necessary for a physically meaningful comparison between model classes.
- This is one of the main reasons why the benchmark remains close to later embedded deployment: the estimators are not evaluated as offline regressors, but as causal state estimators under measurement-side constraints.

6.9 Baseline Performance

- Under nominal baseline conditions, the hybrid direct-measurement model achieves the lowest error on the selected benchmark trajectory.[2]
- The reported baseline metrics are: DM with MAE 0.0311, RMSE 0.0397, P95 error 0.0800, and bias 0.0311; HDM with MAE 0.0024, RMSE 0.0061, P95 error 0.0094, and bias 0.0022; HECM with MAE 0.0090, RMSE 0.0127, P95 error 0.0241, and bias -0.0029; DD with MAE 0.0100, RMSE 0.0143, P95 error 0.0336, and bias -0.0062.
- This ranking is important because it shows immediately that the clean-condition leader is not the same as the overall robustness leader in the disturbed scenarios that follow.
- The baseline results therefore establish the necessary reference point for interpreting error growth, recovery thresholds, and scenario-specific ranking changes, but they do not already determine the final deployment-oriented ranking.

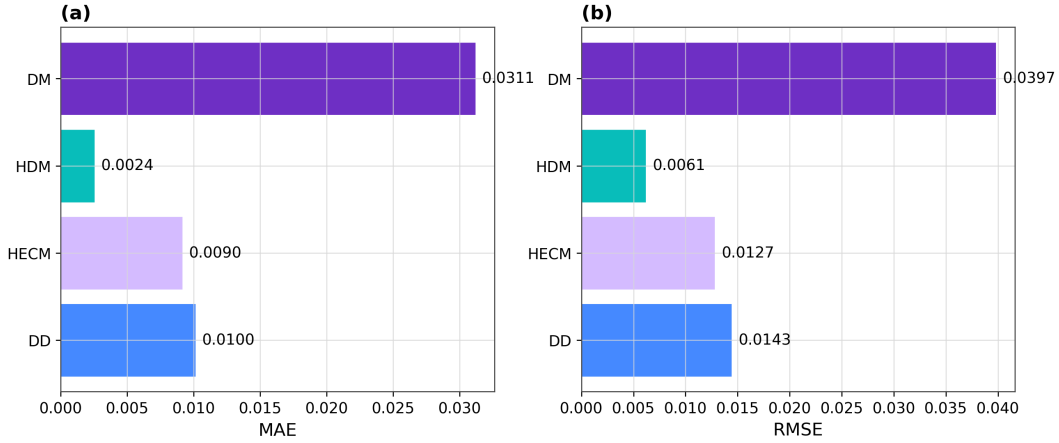


Abbildung 6.2: Baseline performance on the selected full-cell benchmark trajectory. The hybrid direct-measurement model provides the lowest clean-condition error, whereas the direct-measurement model shows the largest baseline deviation.

6.10 Current-Bias and Noise Sensitivity

- Current bias is one of the most discriminative scenarios in the entire benchmark.[2]
- Across the matched current-bias sweep of 0.5%, 1.5%, and 3.0% of $|I|_{\max}$, HECM is the strongest estimator, DD is intermediate, and the two direct-measurement variants degrade strongly because persistent bias accumulates directly through the integration path.
- In the reported current-bias scenario, HECM reaches MAE 0.0932, compared with 0.2859 for DD, 0.3068 for HDM, and 0.3143 for DM.
- The corresponding P95 errors are 0.2954 for HECM, 0.6367 for DD, 0.6668 for HDM, and 0.6670 for DM, which shows that the difference is not only an average-effect artifact.
- This result makes the structural distinction between integration-driven and correction-capable estimators directly visible: explicit voltage-based correction suppresses the long-term drift that dominates the direct-measurement variants.
- Current noise is weaker than current bias in terms of global MAE, but it remains informative through local behavior and output variability.

- Under the high-noise case with $\sigma_I = 0,10$ A, the MAE changes are -0.0001 for DM, +0.0019 for HDM, +0.0075 for HECM, and +0.0009 for DD. HECM therefore shows the largest global penalty in this scenario.
- The local noise story differs from the global metric. In the appendix metrics, the mean excess rolling MAE is -0.0001 for DM, 0.0019 for HDM, 0.0073 for HECM, and 0.0009 for DD. This means that HECM is most sensitive in the error domain, whereas DD shows strong disturbance propagation through several derived input channels.
- Voltage noise with $\sigma_U = 0,01$ V is one of the strongest discriminators of the entire benchmark. HECM remains essentially unchanged with $\Delta\text{MAE} \approx 0$, whereas DM increases by 0.0213, HDM by 0.0727, and DD by 0.0597.
- Temperature noise with $\sigma_T = 1,0$ °C is almost neutral for DM and HECM, but increases MAE by 0.0043 for HDM and by 0.0081 for DD.
- Read together, these results show that disturbance relevance depends strongly on the coupling between the corrupted measurement channel and the internal correction logic of the estimator.

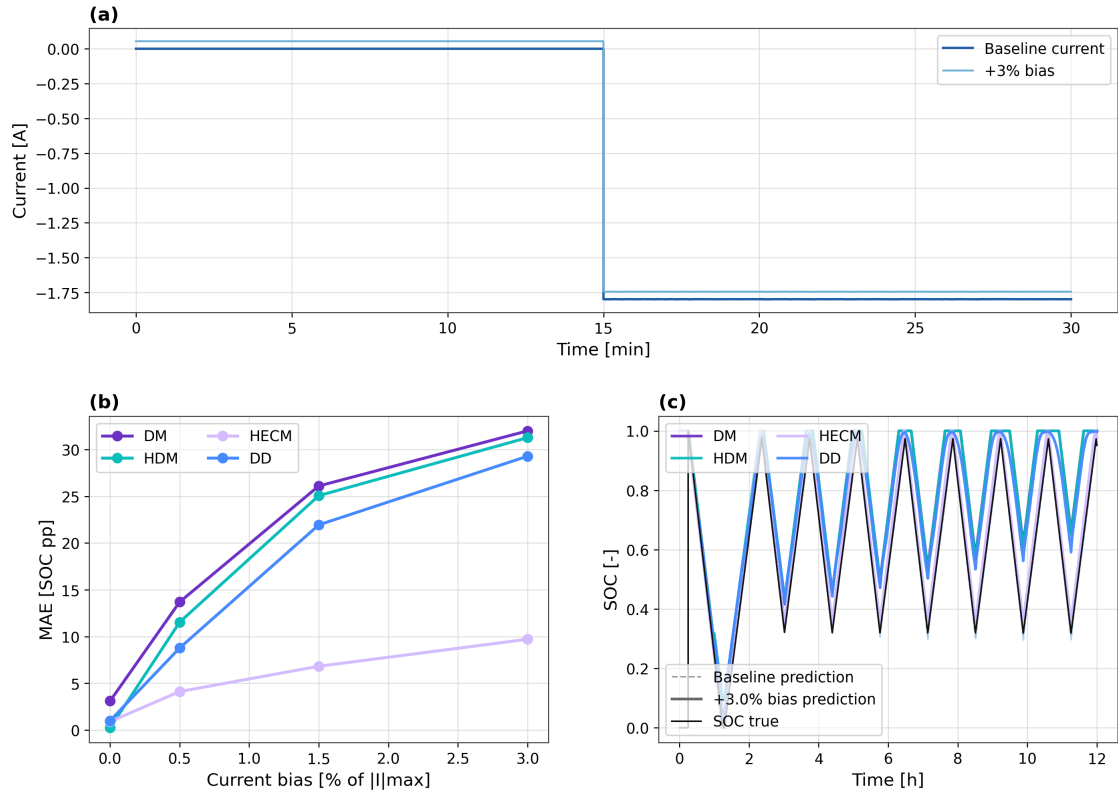


Abbildung 6.3: Sensitivity to current-measurement bias, including a representative disturbed window, the bias sweep, and the resulting SOC divergence over time.

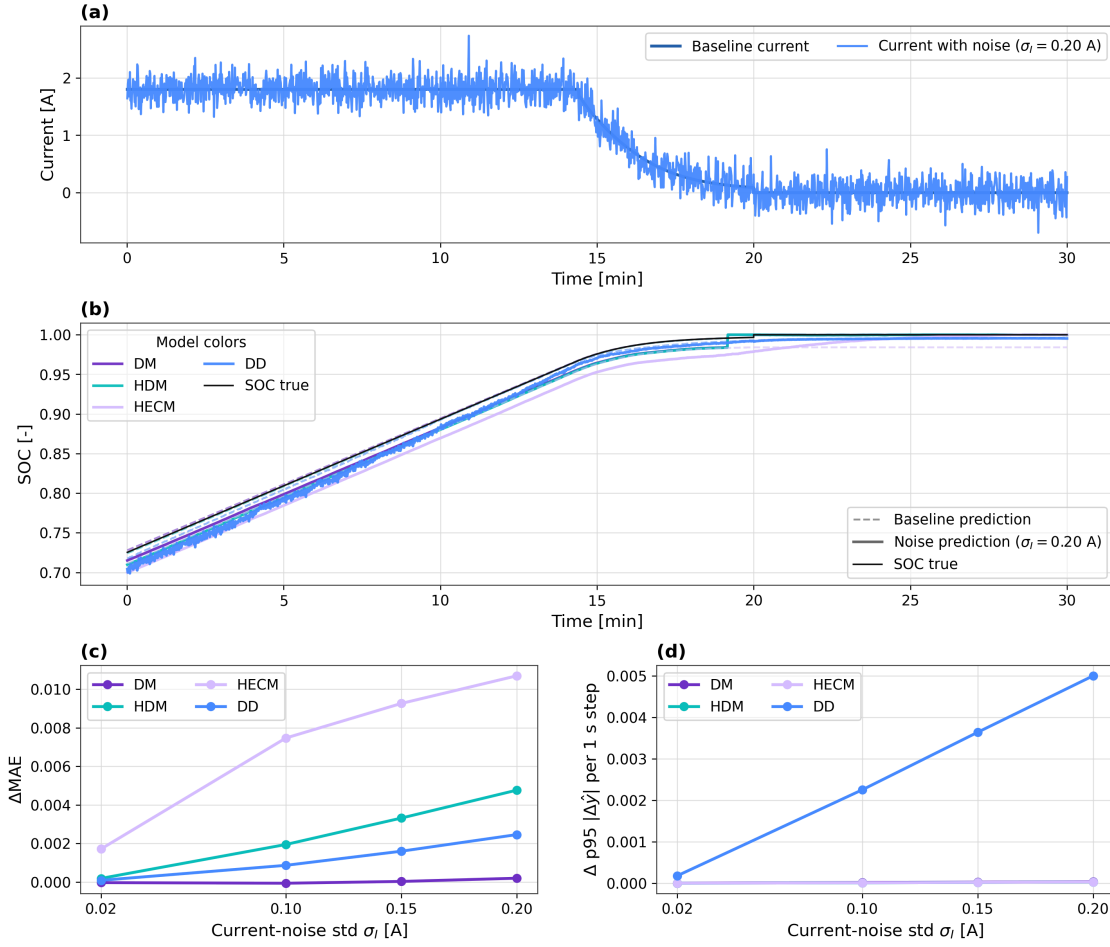


Abbildung 6.4: Noise sensitivity under additive current noise, including a representative disturbed window, global MAE penalties, and local output-variability effects.

6.11 Initialization Mismatch and Recovery

- Initialization mismatch must be interpreted locally, because global MAE hides the actual resynchronization behavior after a wrong initial state.[2]
- The benchmark injects an initial SOC mismatch of -0.10 . For DD, the disturbance is implemented as a perturbation of the online cumulative-charge feature rather than as an artificial latent-state reset, which keeps the test aligned with deployment-time observables.
- The strict recovery thresholds are 0.0800 for DM, 0.0094 for HDM, 0.0241 for HECM, and 0.0336 for DD. The fair thresholds are 0.1599 for DM, 0.0200 for

HDM, 0.0482 for HECM, and 0.0672 for DD.

- Under the strict criterion, DM returns after 1.1658 h, HECM after 2.3693 h, and DD after 1.0464 h. HDM does not re-enter the strict band in the inspected window.
- Under the fair criterion, DM returns after 0.2500 h, HECM after 1.1564 h, and DD after 1.0014 h, while HDM again does not re-enter in the inspected window.
- This recovery view reveals a ranking that differs from both the clean baseline and the current-bias scenario. An estimator can therefore be strong under one deployment-relevant failure mode and weaker under another.

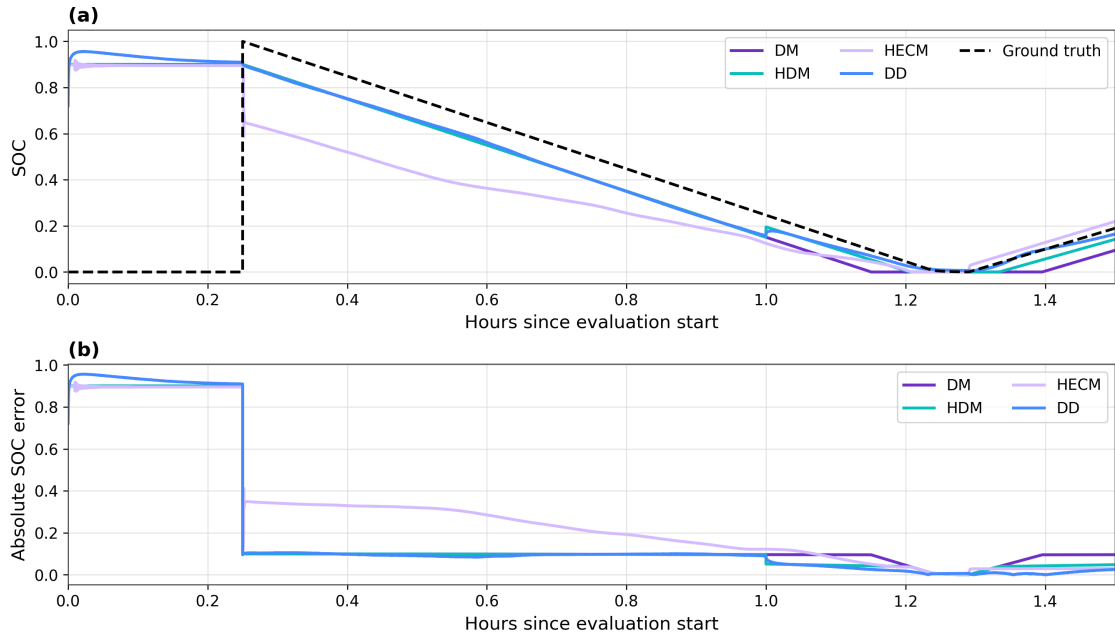


Abbildung 6.5: Recovery after initialization mismatch in a reset-free evaluation window, showing the disturbed SOC trajectories and the corresponding absolute SOC error over time.

6.12 Missing Samples, Timing Faults, and Signal Integrity

- Missing samples are the strongest global signal-integrity stressor in the benchmark.[2]

- When every 50th sample is frozen, the resulting MAE values are 0.0386 for DM, 0.0103 for HDM, 0.0119 for HECM, and 0.0117 for DD. Relative to baseline, this corresponds to $\Delta\text{MAE} = 0.0076$ for DM, 0.0079 for HDM, 0.0029 for HECM, and only 0.0016 for DD.
- The irregular-sampling scenario with uniform timing jitter of ± 0.1 s around the nominal 1 s grid remains nearly neutral for all estimators. The reported MAE changes are -0.0001 for DM, +0.0003 for HDM, +0.0004 for HECM, and +0.0005 for DD.
- Burst dropout is implemented as one central frozen gap of 3600 s. The global MAE values remain moderate at 0.0332 for DM, 0.0051 for HDM, 0.0104 for HECM, and 0.0122 for DD, but the post-gap recovery is long for all estimators.
- The reported recovery times after burst dropout are 27.3511 h for DM, 27.4093 h for HDM, 28.5611 h for HECM, and 27.4351 h for DD.
- These results show that signal-integrity faults require both global and local evidence. A model may remain acceptable in full-run MAE while still exhibiting a practically relevant recovery problem after a large frozen-data window.
- For deployment interpretation, the signal-integrity block is particularly important because it connects algorithm robustness to realistic logging, communication, and task-scheduling failures that occur in embedded systems.

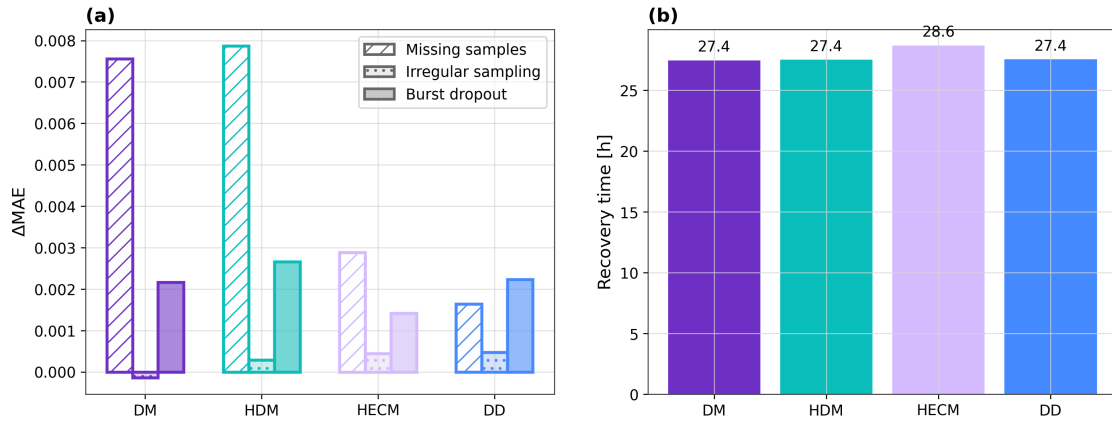


Abbildung 6.6: Signal-integrity summary comparing the global MAE penalties for missing samples, irregular sampling, and burst dropout together with the burst-dropout recovery behavior.

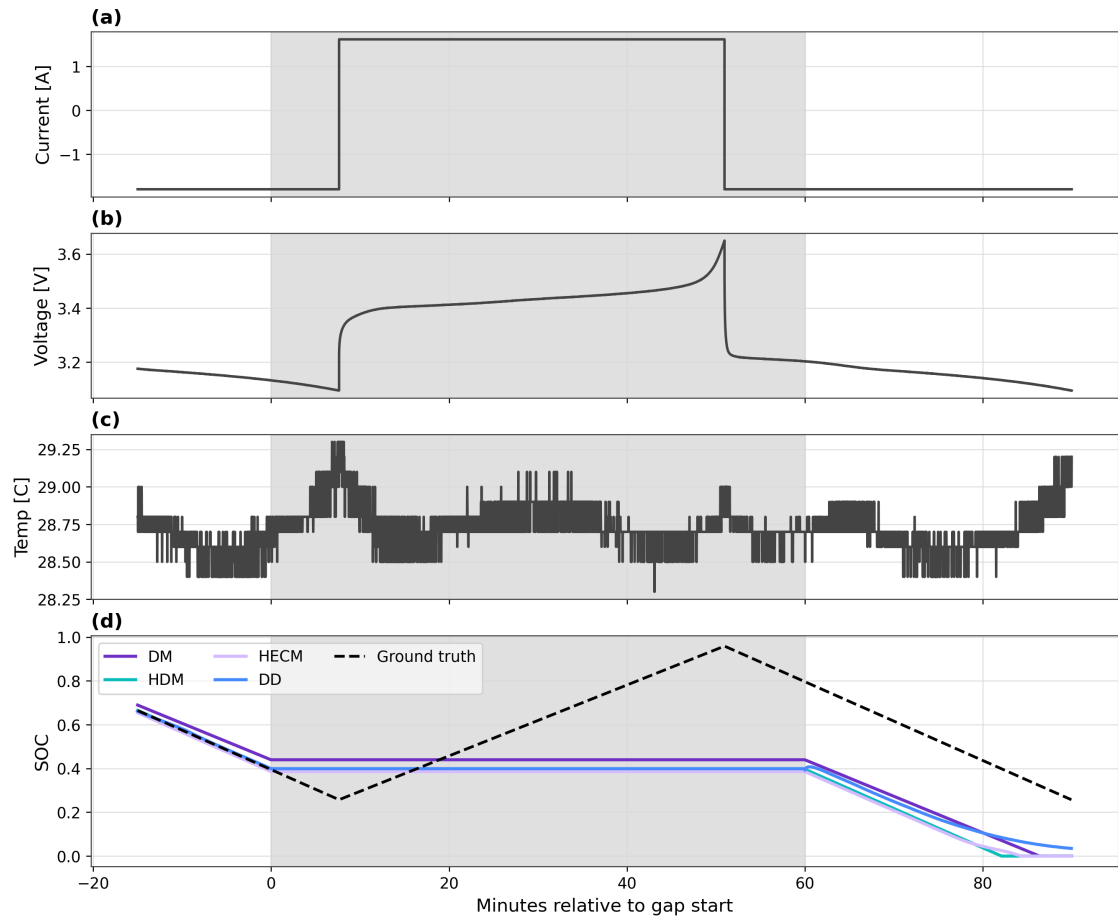


Abbildung 6.7: Local transition into the burst-dropout interval. The figure shows how the estimators behave while the input stream is frozen.

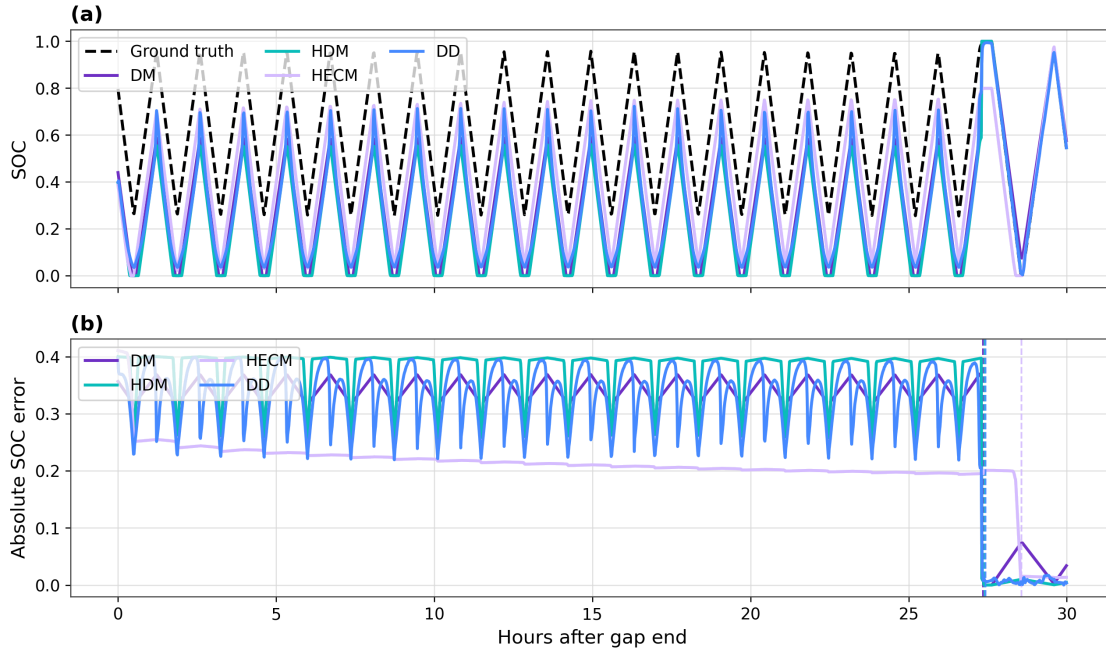


Abbildung 6.8: Post-gap recovery after burst dropout, including the disturbed SOC trajectories and the corresponding absolute SOC error after the frozen interval.

6.13 Voltage-Spike Sensitivity and Cross-Scenario Comparison

- Single voltage spikes lead to strongly different local reactions across the estimator classes, even when global MAE changes remain small for some models.[2]
- The spike protocol injects one $\pm 0,20$ V single-sample spike every 1000 samples.
- The resulting MAE values are 0.0312 for DM, 0.0062 for HDM, 0.0090 for HECM, and 0.0144 for DD. Relative to baseline, the strongest increase appears in DD with $\Delta\text{MAE} = 0.0044$, followed by HDM with 0.0038.
- HECM remains essentially unchanged under this protocol, whereas DD shows the clearest spike sensitivity both in representative local windows and in aligned post-spike error behavior.
- The compact cross-scenario comparison confirms that no single estimator dominates all criteria. HDM is the leader under clean conditions, HECM is strongest

under current bias and voltage noise, and DD is strongest under missing samples and initialization recovery.

- Current bias and voltage noise are the most discriminative scenarios in the present benchmark phase because they separate correction-capable estimators most clearly from purely integration-driven and fully data-driven alternatives.
- ADC quantization remains comparatively weak in this benchmark and is therefore interpreted as part of the total scenario space, but not as one of the strongest ranking-changing disturbances.
- The cross-scenario view is therefore essential for decision-making: it reveals where rankings change across disturbance classes and where estimator strengths are scenario-specific rather than universal.

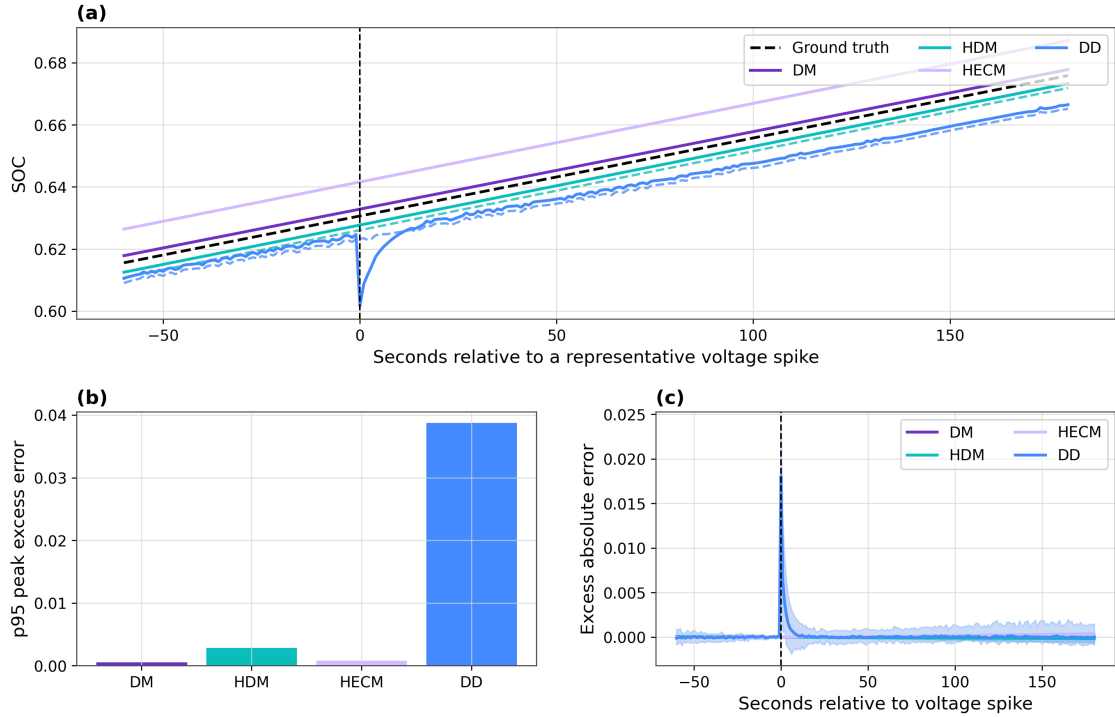


Abbildung 6.9: Voltage-spike sensitivity, including a representative local response, the spike-severity summary, and the aligned post-spike excess-error behavior.

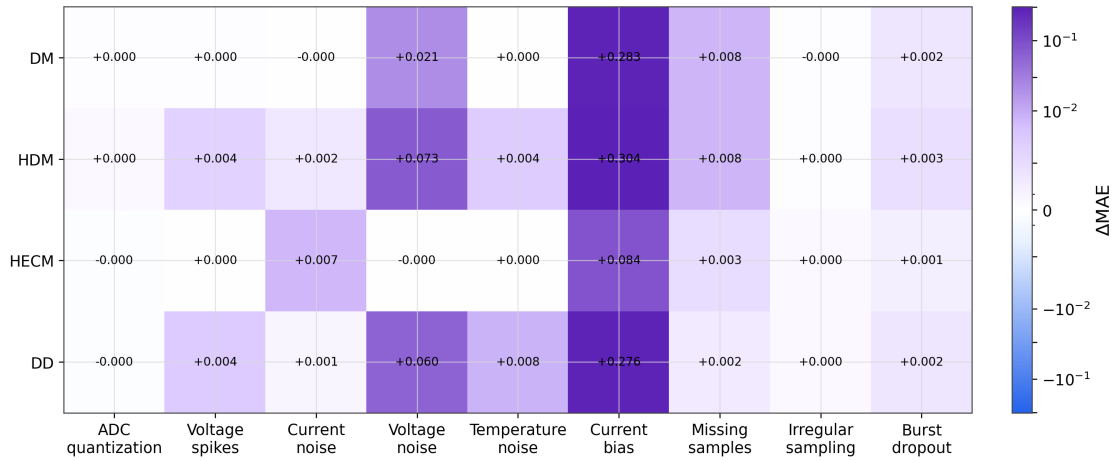


Abbildung 6.10: Cross-scenario error increase relative to the baseline case. The heatmap condenses the disturbed-scenario penalties and makes estimator-specific ranking shifts visible across disturbance classes.

6.14 Interpretation of the Robustness Results

- The direct-measurement model behaves as the expected drift-prone reference. Persistent current disturbances accumulate directly into the SOC estimate, and missing information cannot be corrected through an additional feedback path.[2]
- HDM improves nominal calibration by adapting the effective capacity through online SOH support, but it remains structurally exposed to the same current-integration failure mode and additionally inherits the dynamics of the learned SOH assistance.
- HECM benefits from interpretable parameterization, explicit voltage-residual correction, and causal observer structure. This explains why it is the overall robustness leader in the most important current-bias and voltage-noise scenarios.
- DD behaves as a compact context-rich estimator with channel-dependent strengths and weaknesses. It is especially strong under missing samples and initialization mismatch, but more exposed to voltage spikes and to noise propagation through derivative-related features.
- A central insight of the chapter is therefore that low clean-condition MAE does not imply strong disturbed-operation performance. The trade-off is not only

between accuracy and computational complexity, but also between calibration strength, correction capability, recovery speed, and disturbance propagation.

- The benchmark also shows that estimator comparison should not collapse everything into one leaderboard. A deployment-oriented comparison must separate nominal accuracy, disturbance robustness, recovery after inconsistency, and implementation feasibility.

6.15 Decision-Oriented Deployment Interpretation

- If lowest nominal SOC error under clean sensing is the main design priority, HDM is the recommended class because it achieves the lowest baseline MAE of 0.0024. The corresponding trade-off is strong sensitivity to current bias and the lack of recovery to the baseline bands in the inspected initialization-mismatch window.[2]
- If the primary priority is best overall robustness under uncertain measurement quality, HECM is the strongest choice because it is most resilient under current bias, remains nearly unchanged under voltage noise, and still preserves competitive baseline accuracy. The trade-off is the larger flash footprint and slower strict-band recovery than DD.
- If fastest resynchronization after restart or wrong initial state dominates the design, DD is the strongest option because it shows the fastest strict-band recovery in the local initialization analysis. The trade-off is greater sensitivity to voltage spikes and weaker performance than HECM under current bias.
- If resilience to missing samples and signal freezing dominates the design, DD again becomes attractive because it shows the smallest MAE increase under missing samples and strong post-disturbance stability. The trade-off is that clean-condition accuracy remains below HDM and voltage-driven disturbances require a careful feature-pipeline design.
- If the main objective is maximum implementation simplicity and the largest resource margin, DM remains the simplest class by far, but it also shows the weakest robustness under biased current measurements and missing-sample stress.

- The practical message of the chapter is therefore not that one estimator wins universally. The message is that estimator selection must be tied to the disturbance profile, recovery requirements, and implementation constraints of the target BMS.

6.16 Limitations and Future Extensions

- The current benchmark phase is based on one representative dataset trajectory and should later be extended to additional cells, broader ageing states, and cross-cell validation before stronger generalization claims are made.[2]
- The spike study currently uses isolated spike events. Clustered disturbances, longer spike sequences, and hardware-level disturbance injection may reveal additional model differences.
- The initialization test for DD remains an online-state proxy based on Q_c perturbation rather than an explicit latent-state reset. This is methodologically justified, but it should still be interpreted as a class-specific approximation.
- Future extensions should include the remaining extended scenarios, additional cells, pack-level settings, and eventually hardware-in-the-loop or true embedded runtime benchmarks under the same performance-oriented evaluation philosophy.

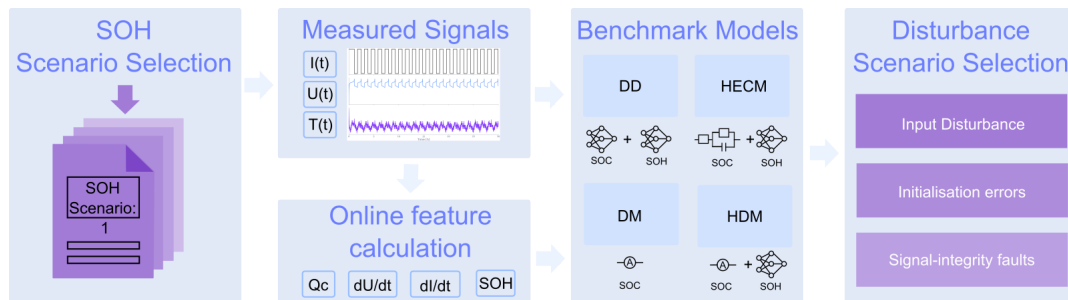


Abbildung 6.11: Methodology overview of the robustness benchmark, including disturbance injection, matched estimator evaluation, and comparison logic.

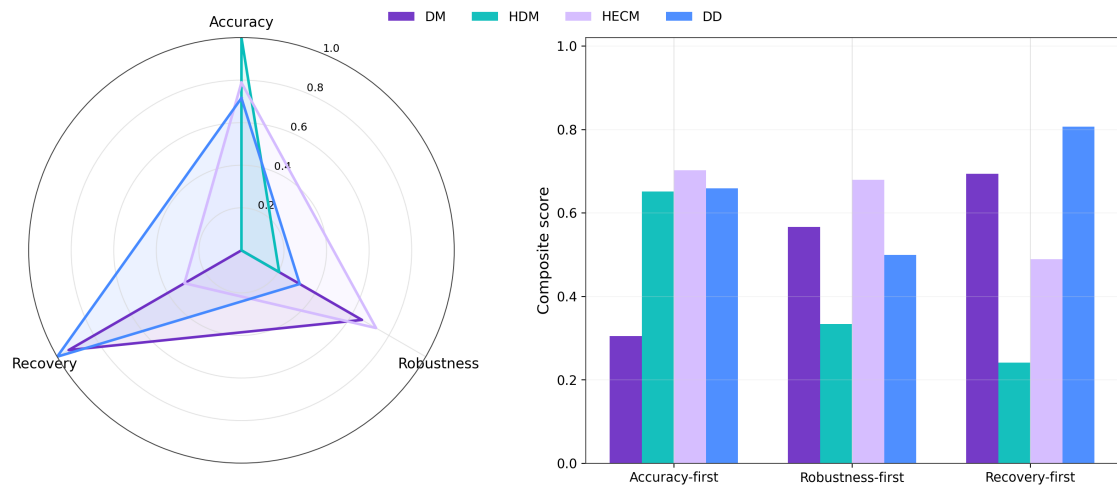


Abbildung 6.12: Decision-oriented synthesis of the robustness benchmark showing how estimator-selection guidance can be derived from disturbance-specific results.

Embedded Artificial Intelligence for SOC and SOH Estimation on Microcontrollers

7.1 Embedded Deployment Objective

- This chapter examines how SOC and SOH estimators can be transferred from floating-point research models to practical microcontroller execution.[3]
- The central contribution lies in the complete deployment path from trained recurrent models to executable MCU firmware using structured pruning, INT8 weight quantization, manual export, and on-device benchmark measurements on an STM32H753ZI.
- In contrast to purely algorithmic model comparisons, the focus here is on the interaction between predictive utility, resource demand, and implementation effort on the target hardware.
- The paper therefore treats battery AI explicitly as an embedded systems problem: an estimator is only useful if it remains accurate under long-horizon streaming replay and still fits into the flash, SRAM, runtime, and energy budget of the controller.
- The chapter follows the paper closely and transfers the original logic almost one-to-one: dataset basis, feature engineering, stateful LSTM architecture, pruning, quantization, firmware export, STM32 benchmarking, and the resulting multi-objective trade-offs.

7.2 Dataset Basis and Experimental Scope

- All experiments rely on the published LFP DoE Cycle Ageing Dataset, which was created at Technische Universität Berlin on commercially available JGCFR18650-1800mAh-3.2V LFP cells.[3]
- The ageing campaign follows a design-of-experiments scheme in a climatic chamber at 25 °C. The three varied factors are the charge rate $C_{\text{chg}} \in [0.5 \text{ C}, 0.9 \text{ C}]$, the discharge rate $C_{\text{dis}} \in [1.0 \text{ C}, 2.5 \text{ C}]$, and the depth of discharge DoD $\in [45\%, 65\%]$.
- A mixed full/fractional factorial plan assigns 15 test cells to eight operating points, with duplicate conditions for reproducibility. Between ageing loops, dedicated check-ups with capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation are inserted.
- In addition to controlled cycling, stationary-storage load profiles derived from residential PV home storage are injected so that the data contain both laboratory diagnostics and application-oriented operating conditions.
- The raw measurements are available at 1 Hz and contain voltage, current, terminal temperature, and auxiliary channels. This combination is important because it exposes the estimators to both fast SOC transients and slow SOH evolution across long ageing horizons.

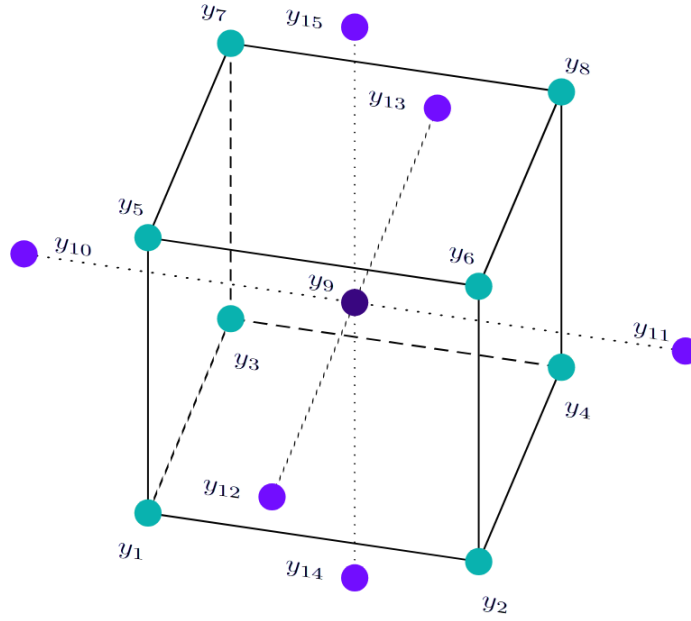


Abbildung 7.1: Design-of-experiments cube of the LFP ageing dataset used in the embedded AI paper.

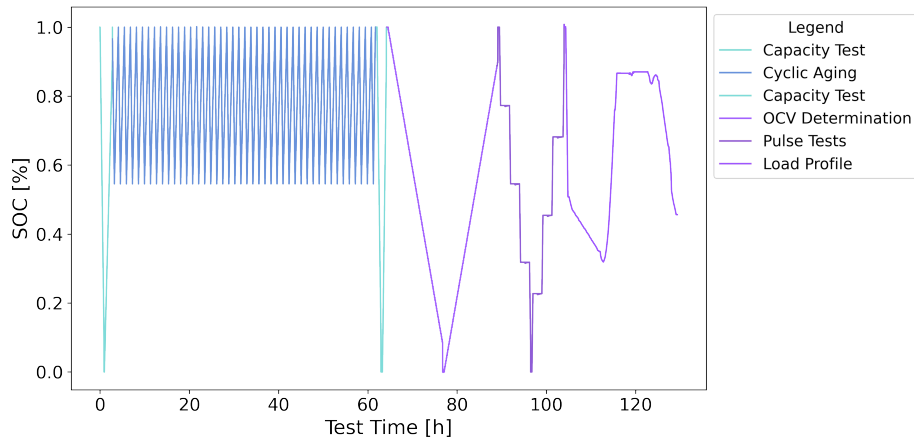


Abbildung 7.2: Representative SOC trajectory showing cyclic ageing, check-up blocks, and stationary-storage load-profile segments.

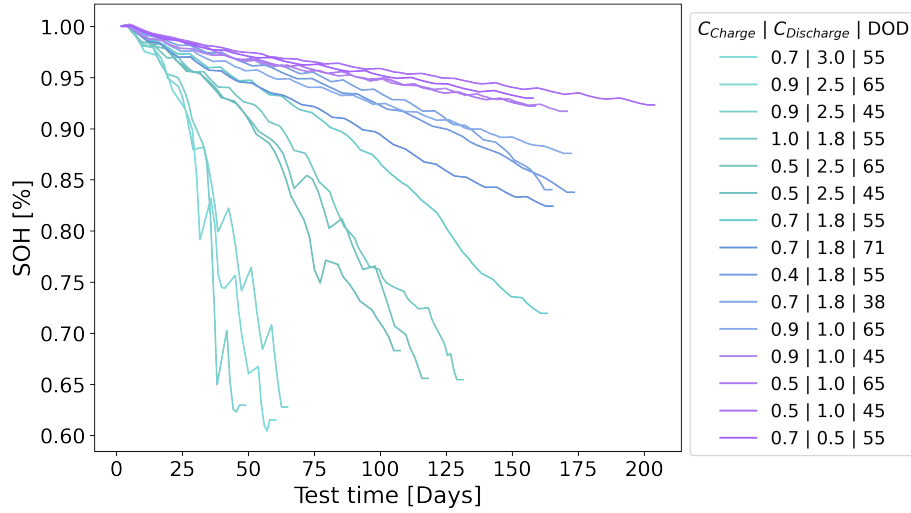


Abbildung 7.3: SOH progression across the LFP ageing campaign, illustrating the broad spread between cells and operating points.

7.3 Feature Engineering and Reference Targets

- The SOC estimator uses the six-dimensional feature vector $[U(t), I(t), T(t), Q_c(t), dU/dt, dI/dt]$, where the temporal derivatives are computed by centred finite differences.[3]
- Each SOC feature is robustly normalised with median and interquartile range so that the exact same scaling statistics can be embedded into the firmware.
- The SOH estimator uses a different feature stack that reflects the slower degradation dynamics: $[t, U(t), I(t), T(t), EFC(t), Q_c(t)]$, where $EFC(t)$ denotes equivalent full cycles.
- The SOC reference is derived from Coulomb counting with drift compensation. After each constant-voltage termination the integral is reset to $SOC = 1$, which reduces long-term bias caused by capacity fade.
- The SOH reference is defined from periodic capacity check-ups as C_k/C_{nom} , that is, the measured discharge capacity relative to nominal capacity.
- Both reference traces are explicitly offline constructs. They rely on information that is not available in deployment, such as long rest periods for OCV stabilisation or periodic full-capacity measurements. On the STM32 the estimators operate only on streamed measurements and internal states.

- The paper then uses a strictly stateful streaming regime: hidden and cell states are propagated sample by sample along the full sequence, without sliding windows. Host-side replay and STM32 replay therefore follow identical inference conditions.

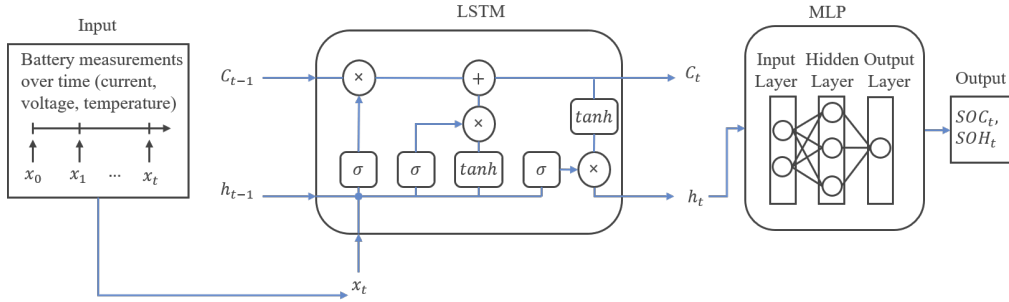


Abbildung 7.4: Stateful LSTM + MLP structure used for both SOC and SOH estimation in the embedded paper.

7.4 LSTM Architecture and Training Setup

- Both SOC and SOH estimators employ the same recurrent equation set, but with task-specific size. The SOC model uses an LSTM with hidden size $H = 64$ and an MLP output head of width 64, whereas the SOH model uses hidden size 128 and an MLP width of 128.[3]
- This restriction to a single-layer LSTM + MLP architecture is deliberate. The paper argues that the main research focus is not broad architecture search, but a transparent and reproducible embedded tool flow for stateful estimators.
- Training is carried out on NVIDIA A100-class GPU hardware in mixed precision (FP16). Both tasks use the AdamW optimiser, cosine-annealing warm restarts, gradient clipping, and mini-batch training.
- For SOC, the learning rate is 1.5×10^{-4} , the batch size is 512, and the sequence chunk length is 2024. For SOH, the learning rate is 2.0×10^{-4} , the batch size is 128, and the chunk length is 2048.
- Although deployment is fully stateful and step-wise, training uses truncated backpropagation through time over these chunks. Within each chunk, the hidden and cell states are propagated exactly as in deployment so that the stateful update mechanism is already learned during optimisation.

- The abstracted performance envelope reported by the paper is already informative: over long-horizon streaming replay, the three SOC variants achieve MAE values between 2,3 % and 2,8 % of full scale, while the SOH variants reach MAE values between 0,9 % and 1,5 %.

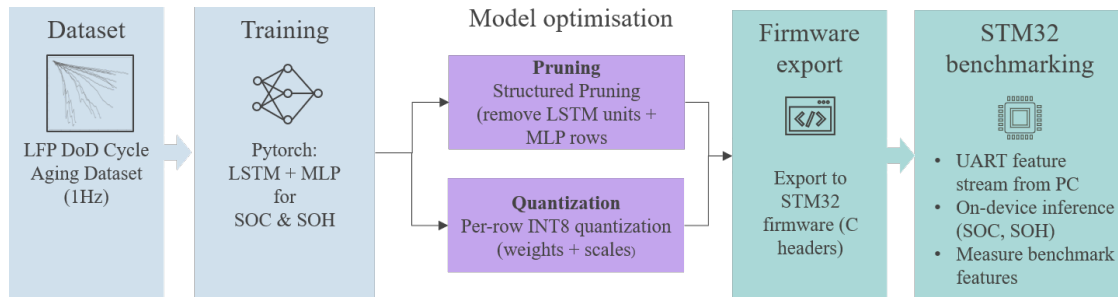


Abbildung 7.5: End-to-end deployment pipeline from dataset and PyTorch training to header export and STM32 benchmarking.

7.5 Structured Pruning for MCU Deployment

- Pruning is applied as structured magnitude pruning at the level of complete LSTM hidden units.[3]
- The saliency score aggregates the L2 norms of the incoming weights for each hidden unit across all four LSTM gates. Hidden units with the smallest scores are removed first.
- In the final deployment, the SOC estimator is reduced from 64 to 45 hidden units and the SOH estimator from 128 to 90 hidden units. In both cases this corresponds to roughly 30% channel-wise sparsity.
- Rows and columns are removed jointly from the recurrent gate matrices and the downstream MLP weights so that the pruned network remains a valid stateful recurrent mapping.
- After each pruning step the model is fine-tuned again. This is important because the paper does not treat pruning as a purely static compression step, but as a repeated prune-and-recover cycle.
- The engineering purpose of pruning is direct: reduce recurrent state size, shrink flash and RAM demand, and shorten on-device inference time without materially compromising streaming accuracy.

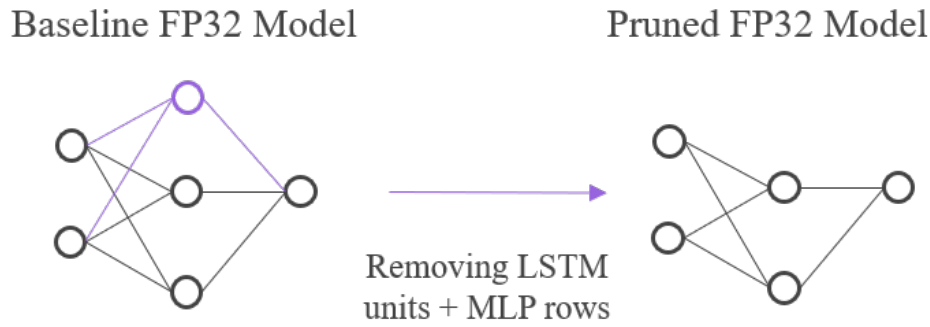


Abbildung 7.6: Structured pruning concept used in the embedded AI paper.

7.6 INT8 Quantization and Manual STM32 Export

- Quantization complements pruning by reducing the numerical precision of the remaining weights while leaving the topology unchanged.[3]
- The paper uses post-training per-row symmetric INT8 quantization for the input-to-gate and hidden-to-gate matrices of the LSTM. Each row receives its own floating-point scale factor, while biases and activations remain in FP32.
- This design choice is pragmatic: it preserves numerical stability and keeps the firmware implementation close to the PyTorch reference while still reducing recurrent weight storage by roughly a factor of four relative to FP32 storage.
- The embedded gate computation reconstructs the effective weights from INT8 codes and FP32 scales during inference. A built-in streaming comparison validates the conversion and typically yields MAE differences below 10^{-4} between exported and reference implementations.
- The exported model parameters are written as C headers and integrated into manually implemented stateful LSTM + MLP kernels. This manual route is chosen because STM32Cube.AI does not natively support the continuous state handling required for these estimators.
- The result is a fully transparent firmware path: every scaling constant, every recurrent state update, and every arithmetic step can be audited and reproduced outside the proprietary toolchain.

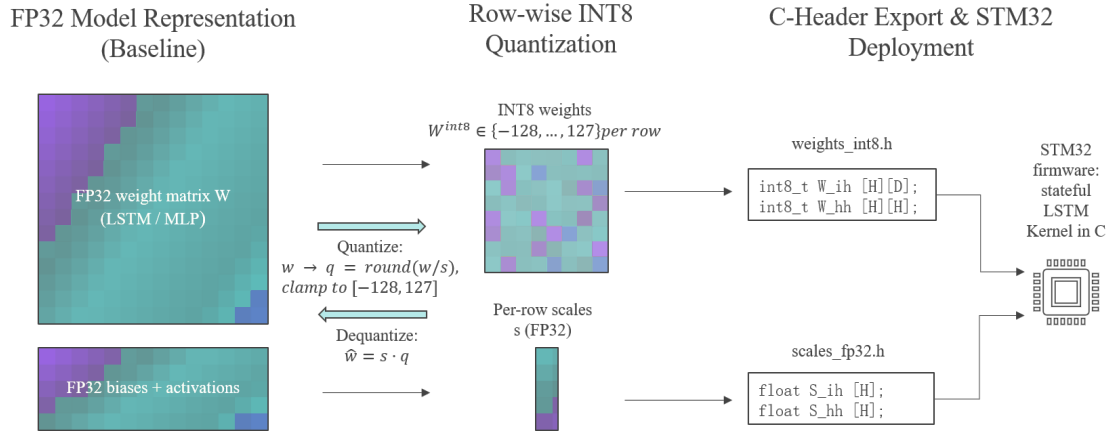


Abbildung 7.7: Per-row INT8 quantization and export concept used for the STM32 deployment.

7.7 Benchmark Methodology and KPI Logic

- The benchmark is executed on an STM32 Nucleo-144 board with an STM32H753ZI microcontroller, providing 2 MB flash and 1 MB SRAM.[3]
- For each deployment, identical feature streams from the LFP dataset are replayed both on the host and on the STM32 so that accuracy, latency, and resource measurements refer to the same stateful input sequence.
- The host-side replay operates directly on the reference PyTorch implementation and produces MAE, RMSE, R^2 , parity views, and long-horizon trajectory comparisons.
- The embedded benchmark injects the same feature vectors via UART, measures end-to-end host latency, measures pure on-device inference time through the DWT cycle counter, and derives energy per inference from the average inference time and an empirically calibrated board power of 0,5 W.
- RAM is computed as the sum of statically allocated ‘.data’ and ‘.bss’ sections plus the maximum observed stack usage. Flash is taken from the linker output of the STM32CubeIDE projects.
- The paper therefore distinguishes carefully between communication-dominated system latency and pure on-device inference time. This distinction matters be-

cause UART framing can dominate the end-to-end measurement although the actual neural computation remains well within the real-time budget.

Tabelle 7.1: STM32 engineering KPIs for the SOC deployments transferred from the embedded AI paper.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	Energy [mJ]
Base FP32	12.70	1.40	105.32	4.93	0.70
Pruned FP32	12.20	0.80	62.27	4.03	0.40
Quant INT8	18.48	6.99	52.48	3.96	3.49

Tabelle 7.2: STM32 engineering KPIs for the SOH deployments transferred from the embedded AI paper.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	Energy [mJ]
Base FP32	34.88	22.73	335.00	8.69	11.37
Pruned FP32	24.69	12.72	182.41	6.96	6.36
Quant INT8	41.23	29.21	138.00	6.70	14.60

7.8 Streaming Prediction Accuracy

- The results section of the paper starts with full-sequence streaming replay over the complete ageing trajectory.[3]
- For SOC, the tracking remains tight across the entire dynamic range. The residuals stay centred around zero, and the quantized model overlaps the full-precision baseline so closely that the two traces are often visually almost indistinguishable.
- The pruned SOC model also remains highly accurate, but shows slightly stronger smoothing in highly dynamic regions. The paper interprets this as the expected effect of the reduced hidden-state dimensionality.
- For SOH, the task is structurally harder because the estimator has to infer slow degradation from sparse check-up events and continuous monitoring data. The paper therefore removes the discrete capacity tests from the estimator input and retains only the remaining check-up blocks and operating segments.

- A causal smoothing stage is applied to the raw SOH predictions. Specifically, the paper uses an exponential moving average with smoothing factor $\alpha = 10^{-6}$ together with a downward-rate limiter of 2×10^{-8} per step to suppress unphysical jumps while maintaining the monotonic degradation trend.
- After this filtering, the pruned and quantized SOH models still track the dense baseline with similar fidelity and preserve the long-term trend despite visible differences in small-scale variability.

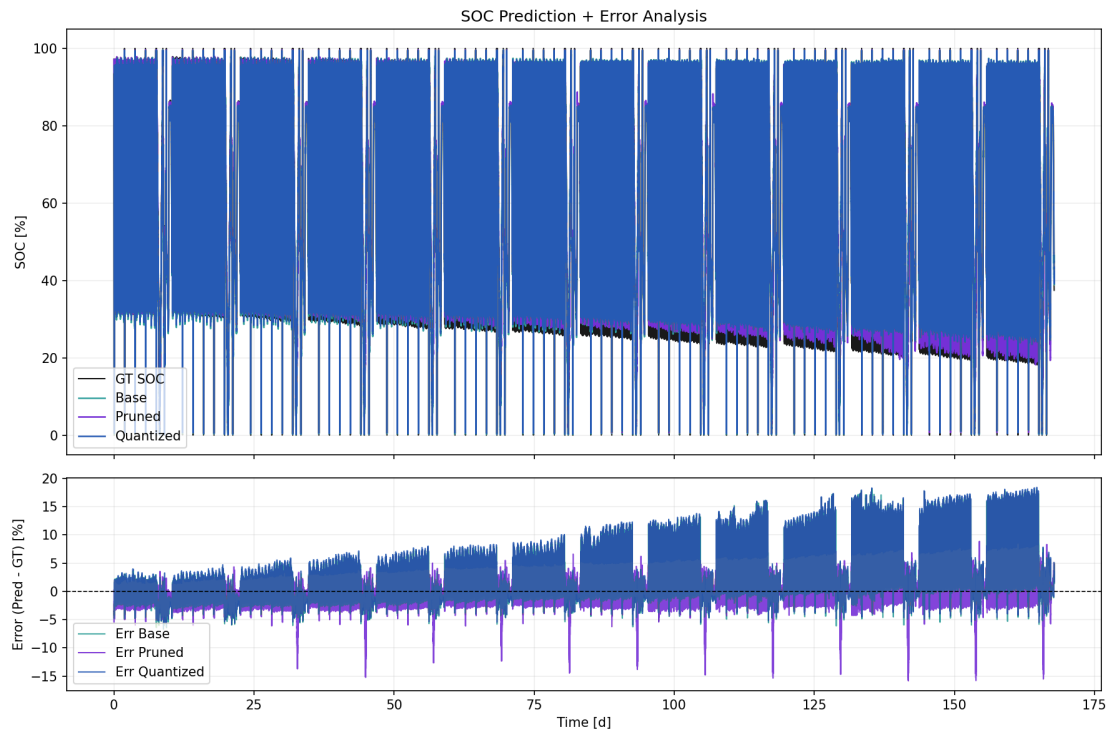


Abbildung 7.8: Streaming SOC dashboard from the embedded AI paper for base, pruned, and quantized deployment.

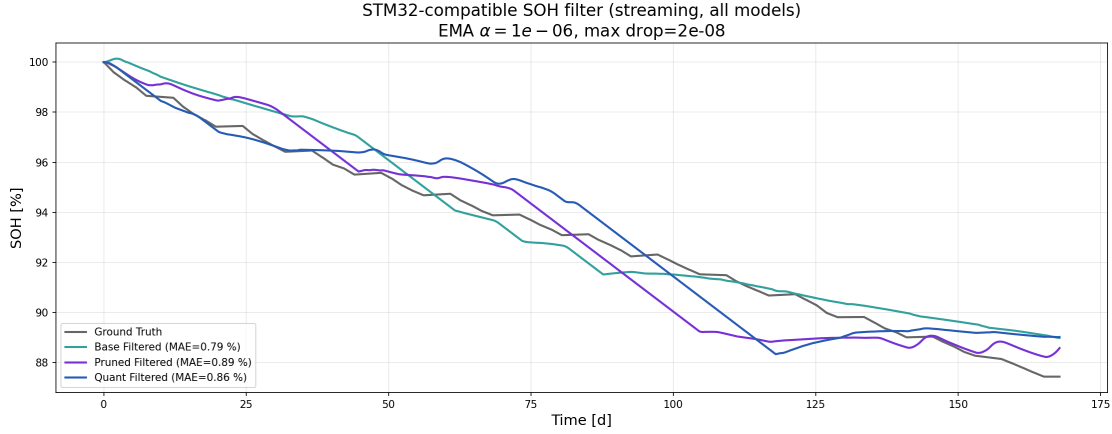


Abbildung 7.9: Streaming SOH dashboard with causal filtering for the three deployment variants.

7.9 Error Distribution and Local Transient Behaviour

- The SOC error distribution is narrow and symmetric, which indicates the absence of systematic bias.[3]
- The paper reports that the pruned SOC model achieves the smallest median MAE and the narrowest interquartile range, which means that pruning slightly improves the SOC task while simultaneously reducing runtime and memory demand.
- For SOH, the error distribution is broader, which is consistent with the greater intrinsic difficulty of estimating capacity fade from operational data. Even so, the bulk of the SOH errors remains within a practically usable tolerance band of approximately $\pm 2\%$.
- To avoid hiding local failures behind global metrics, the paper additionally zooms into a pulse segment between 30k and 40k seconds and into a representative check-up sequence between 657k and 897k seconds.
- In the pulse region, the quantized model follows the reference transients with high fidelity and captures voltage-recovery phases accurately, whereas the pruned model smooths these sharp features slightly.
- In the check-up phase, all models converge well and remain stable over long integration horizons, which confirms that the stateful streaming formulation does

not accumulate visible drift over extended operation.

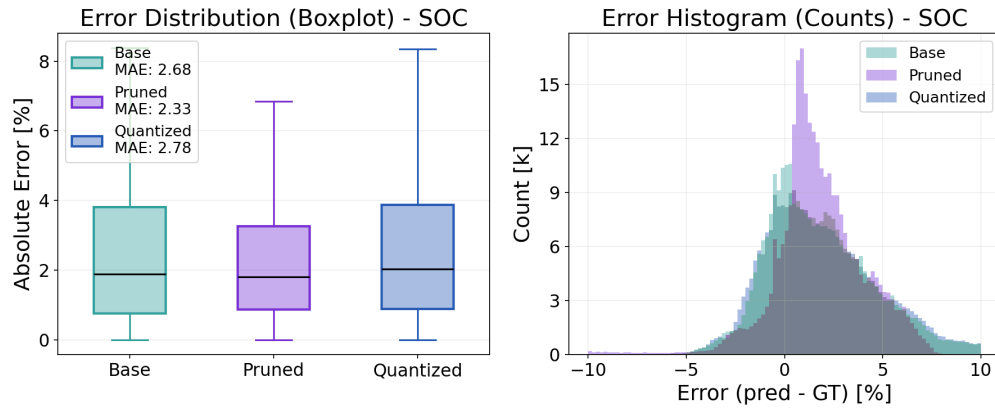


Abbildung 7.10: SOC error distribution from the embedded paper, including MAE boxplots and error histogram.

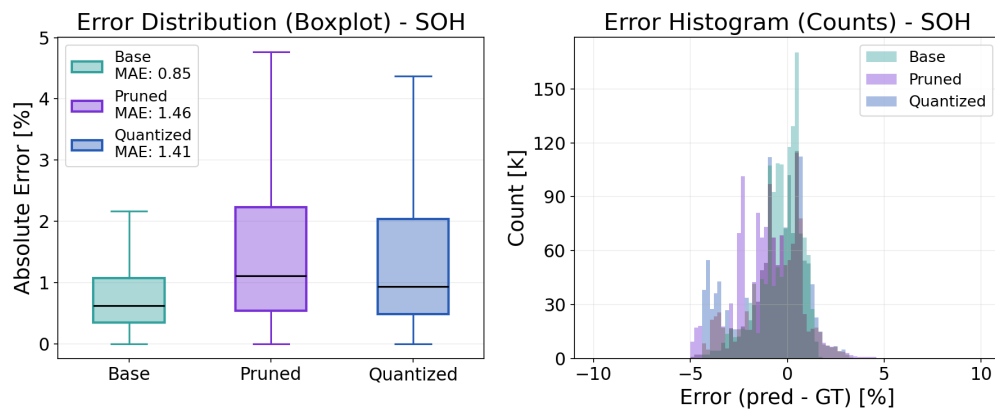


Abbildung 7.11: SOH error distribution from the embedded paper, including MAE boxplots and error histogram.

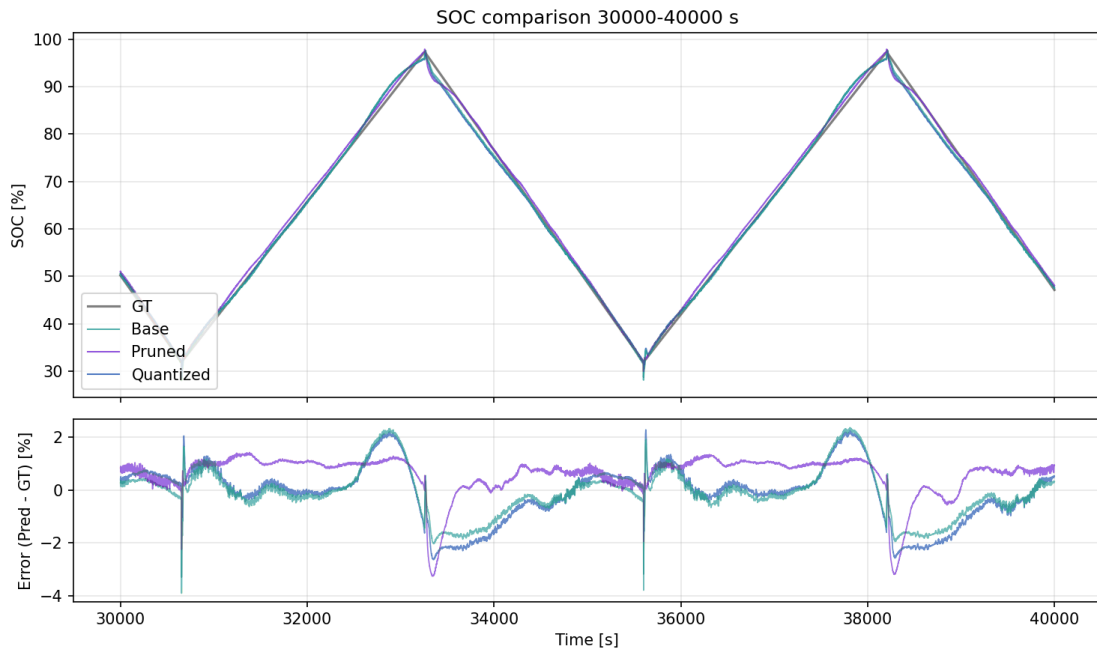


Abbildung 7.12: Local pulse-region analysis of the embedded SOC estimator.

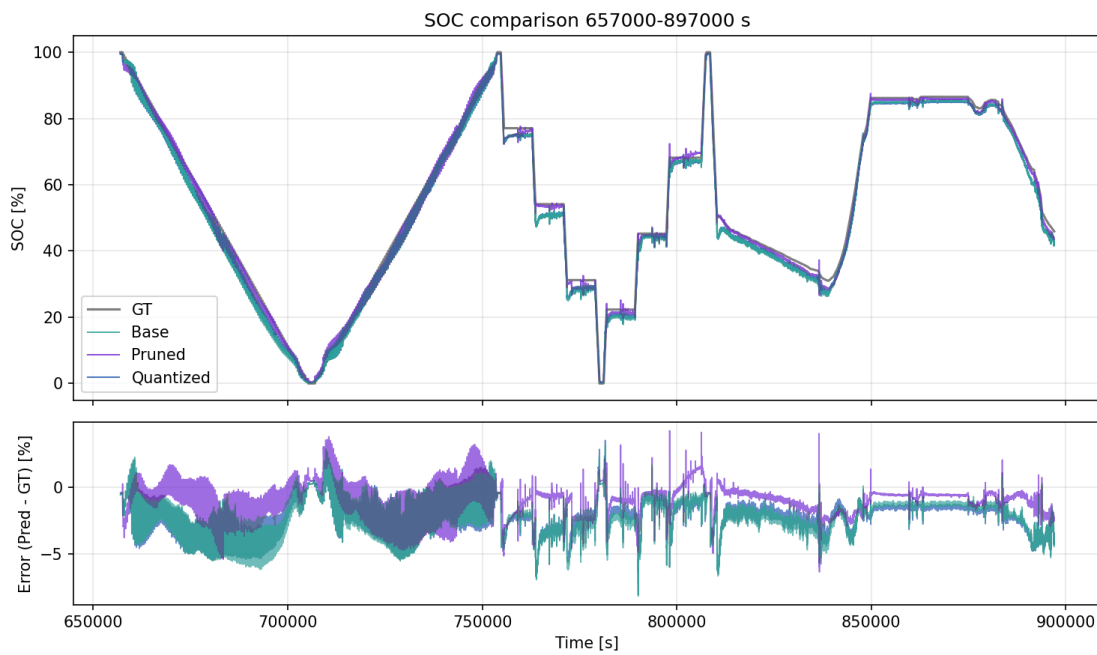


Abbildung 7.13: Local check-up-region analysis of the embedded SOC estimator.

7.10 Resource Efficiency, Latency, and Trade-Off

Summary

- The KPI tables already show that pruning and quantization affect SOC and SOH differently.[3]
- For SOC, pruning reduces flash from 105,32 kB to 62,27 kB, reduces inference time from 1,40 ms to 0,80 ms, and reduces energy per inference from 0,70 mJ to 0,40 mJ. This corresponds to gains of roughly 41–43%.
- Quantization reduces the SOC flash footprint further to 52,48 kB, but increases pure inference time to 6,99 ms and energy per inference to 3,49 mJ because the software kernel has to reconstruct FP32 values from INT8 codes and scales.
- For SOH, pruning reduces flash from 335,00 kB to 182,41 kB, reduces inference time from 22,73 ms to 12,72 ms, and reduces energy per inference from 11,37 mJ to 6,36 mJ. Quantization reduces flash further to 138,00 kB, but raises inference time and energy to 29,21 ms and 14,60 mJ.
- The model-size and latency plots condense the same pattern visually: quantization is the strongest lever for non-volatile memory, while pruning is the strongest lever for energy and inference time on the current hand-written STM32 implementation.
- The paper then introduces a simple composite utility score U that aggregates MAE, flash, RAM, and energy into a single normalised comparison value. By construction, the dense baseline has $U = 1$.

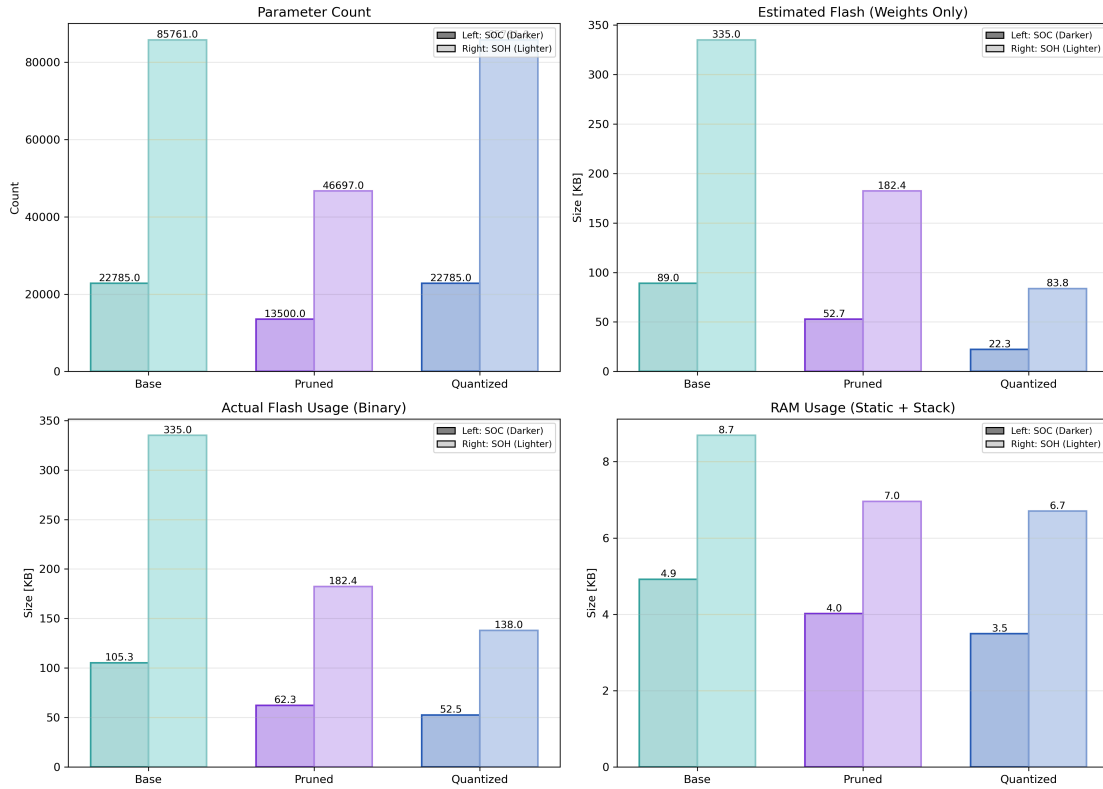


Abbildung 7.14: Resource landscape of SOC and SOH variants, including parameter count, estimated flash, measured flash, and RAM.

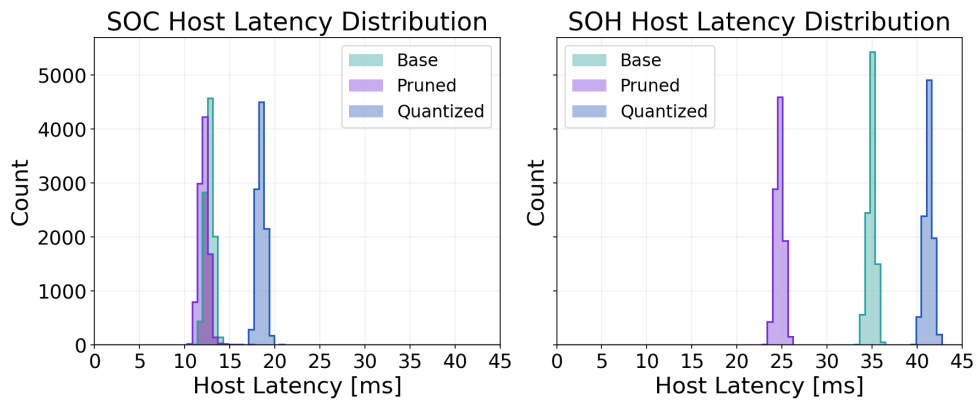


Abbildung 7.15: Latency distributions for SOC and SOH variants on the STM32H753ZI.

Tabelle 7.3: Composite pruning/quantization summary from the embedded AI paper. Values are reported as SOC/SOH.

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔEnergy [%]	U
Pruned	-0.35 / +0.61	-41 / -46	-18 / -20	-43 / -44	0.71 / 0.91
Quantized	+0.10 / +0.56	-50 / -59	-20 / -23	+399 / +29	1.83 / 1.03

7.11 Implications for MCU-Based BMS

- The central engineering outcome of the chapter is that all investigated deployments satisfy the timing constraints of typical stationary-storage BMS applications with comfortable margin.[3]
- Even when communication overhead is included, end-to-end SOC latencies remain below roughly 20ms and SOH latencies below roughly 50ms. The pure compute time on the MCU stays below 1,5ms for dense and pruned SOC and within roughly 12ms bis 30ms for the SOH variants.
- The chapter also makes clear that there is no universally best compression strategy. On the current STM32 implementation, pruning provides the most balanced operating point, whereas quantization is attractive mainly when flash capacity is the dominating constraint.
- This conclusion is reflected directly in the utility score: for SOC, the pruned variant reaches $U = 0.71$ and is therefore the most favourable trade-off, whereas the quantized SOC deployment reaches $U = 1.83$ because the INT8 kernel increases energy strongly. For SOH, both compressed variants remain close to the baseline with $U \approx 0.9\text{--}1.0$.
- From the perspective of the dissertation, this chapter provides the concrete bridge between data-driven estimator design and the actual embedded controller. It shows that SOC and SOH estimators can be co-located on one STM32H7-class platform, but that architecture, compression, and firmware implementation must be selected jointly rather than sequentially.

Architecture-Dependent Effects of Pruning and Quantization

8.1 Problem Statement

- The embedded deployment study already demonstrates that pruning and quantization can improve the feasibility of MCU-based inference.
- The next scientific step is to determine how strongly these gains depend on the underlying estimator architecture.
- This question is important because compression methods that work well for one model family may degrade another model family disproportionately in terms of accuracy, numerical stability, or execution behaviour.
- A cross-architecture analysis therefore extends the embedded perspective from a single deployment pipeline toward a broader model-compression design framework.

8.2 Comparison Framework

- The comparative study varies both estimator architecture and compression strategy under matched embedded evaluation conditions.
- Relevant architecture classes include feed-forward baselines, recurrent models such as GRU or LSTM, and hybrid models with explicit state-related or physics-aware structure.
- Relevant compression variables include structured and unstructured pruning, INT8 quantization, mixed-precision variants, and architecture-specific compression choices.

- The evaluation combines task-specific predictive metrics with Flash, RAM, latency, and numerical-stability indicators so that compression quality is assessed as a joint property of model and hardware.

Tabelle 8.1: Experiment matrix for analysing architecture-dependent pruning and quantization effects under matched embedded conditions.

Model family	Main task	Compression variables	variables	Scientific contribution
MLP	SOH or static surrogate tasks	INT8, mixed precision, structured pruning		Simple low-cost baseline and reference point for interpretability and resource use.
GRU / LSTM	Streaming SOC or SOH	hidden-size pruning, INT8 export, activation range analysis		Direct continuation of the current embedded AI work.
Hybrid neural / physics-aware models	SOC correction or joint estimation	selective compression of neural sub-blocks	com-	Useful for understanding whether hybrid structures compress differently from purely neural ones.
Tiny deploy-ment baselines	MCU-only reference	aggressive latency-first tuning	com- and tu-	Practical lower-bound reference for real BMS integration.

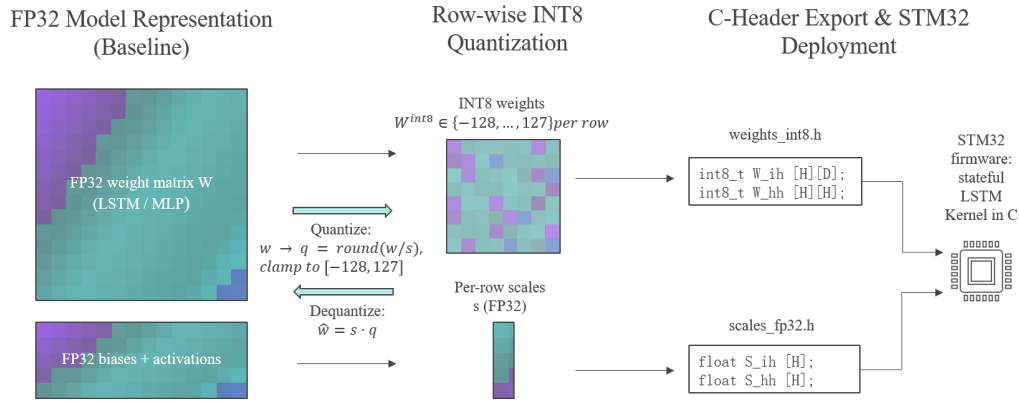


Abbildung 8.1: Quantization workflow used as the methodological starting point for the cross-architecture compression study.

8.3 Scientific Contribution

- The chapter examines which model families retain predictive quality most effectively under aggressive compression and where compression-induced degradation becomes architecture-specific.
- It provides a basis for architecture-dependent design rules that connect estimator structure with quantization and pruning sensitivity.
- In this way, the chapter generalizes the embedded deployment results beyond a single model pipeline and strengthens the hardware-aware interpretation of battery AI.
- The comparative view extends the current embedded work toward transferable deployment guidelines for future BMS integration.

General Discussion and Cross-Paper Synthesis

9.1 Synthesis of the Main Findings

The cumulative findings of the dissertation show that battery-related artificial intelligence cannot be evaluated adequately within a single disciplinary dimension. Model design, disturbance robustness, embedded feasibility, and hardware integration interact directly and must therefore be interpreted together.

- The ageing-estimation chapter shows that relevant battery information can already be captured with a comparatively simple network if the temporal representation and the derived features reflect the underlying electrochemical ageing process.
- The robustness benchmark shows that this representation-oriented view is insufficient on its own, because estimator quality changes once measurement disturbances, timing irregularities, and initialization errors are introduced systematically.
- The embedded-AI chapter shows that the ranking of estimator candidates changes once microcontroller memory, latency, and implementation effort become explicit evaluation criteria in addition to predictive quality.
- The hardware chapter demonstrates that these algorithmic questions are not detached from the physical system. Sensor access, firmware partitioning, estimator exchangeability, and the overall validation chain determine which AI methods can actually be investigated and transferred into BMS-oriented execution environments.

- Taken together, the completed study chapters show that the cumulative scientific gain of the dissertation lies in connecting these layers into one coherent engineering view instead of optimizing only one isolated metric.

9.2 Synthesis Matrix

Tabelle 9.1: Synthesis matrix linking the main contribution chapters to the three central requirement dimensions.

Dissertation contribution	contri-	Performance	Hardware	/	Implementation logic
Neural estimation chapter	ageing-	high relevance for representation quality	indirect relevance	rele-	strong relevance for architecture and feature design
Robustness benchmark chapter	bench-	strongest relevance	medium relevance through sensing realism	rele-	strong relevance for evaluation methodology
Embedded AI chapter	AI chap-	medium relevance	strongest relevance	rele-	strongest relevance for compression and deployment workflow
Architecture-dependent compression chapter	com-	medium relevance	strongest relevance	rele-	strongest relevance for model-hardware co-design
Hardware ecosystem chapter	ecosystem	indirect relevance	strongest relevance	rele-	medium relevance through system integration

9.3 Scientific Overall Contribution

- The dissertation does not merely collect independent studies on battery intelligence. Its core contribution is the combination of representation-oriented modeling, deployment-oriented robustness analysis, embedded implementation, and hardware-grounded validation within one shared research program.
- A second central contribution is the battery-management requirement frame-

work, which makes it possible to compare estimator quality across performance, hardware, and deployment dimensions instead of discussing only one of these layers in isolation.

- A third contribution lies in the methodological connection between offline modelling, disturbance-based benchmarking, and on-device MCU measurements. This link turns the work from a sequence of isolated technical studies into a cumulative framework for deployment-oriented BMS intelligence.
- The overall result is therefore not only a set of chapter-specific findings, but a transferable way of evaluating AI-based battery estimation methods under realistic engineering constraints.

9.4 Methodological Strengths and Limitations Across the Studies

- A key strength of the dissertation is that it spans several layers of evidence: controlled ageing data, structured disturbance benchmarks, measured embedded deployment metrics, and the BMS-STEP hardware platform for practical validation.
- Another strength is the common engineering perspective across otherwise different studies. Ageing estimation, robustness benchmarking, and microcontroller deployment are treated as connected parts of one BMS-related research problem.
- At the same time, the studies rely on different datasets, task definitions, and evaluation settings. Their numerical results are therefore not interchangeable in a strict one-to-one sense, but must be interpreted as complementary evidence for the same overarching design problem.
- The data-driven chapters remain dependent on the breadth and quality of the available datasets. Atypical degradation trajectories, incomplete measurements, and limited coverage of rare operating conditions continue to restrict the generality of the reported results.
- The embedded and hardware-related chapters are likewise shaped by the selected microcontroller targets, firmware environment, and laboratory setup. The

transferability of the exact resource figures to other platforms must therefore be interpreted with appropriate engineering caution.

9.5 Positioning Within the State of Research

- A large part of the existing literature concentrates on one dimension at a time, for example on estimation accuracy, on a particular neural architecture, or on a single deployment toolchain.
- The present dissertation contributes a broader view by showing that battery AI methods must be assessed across model quality, disturbance sensitivity, embedded feasibility, and system integration at the same time.
- In this sense, the work is positioned between algorithm development and embedded systems engineering. It neither treats the battery estimator as a purely offline machine-learning problem nor reduces the hardware side to an implementation appendix.
- This placement is especially important for battery-management applications, because the practical usefulness of an estimator depends on the full chain from measurement acquisition and signal quality to firmware integration and on-device inference behaviour.

9.6 Technical Implications for AI-Based BMS

- The combined results imply that estimator selection for a BMS should begin with the target use case and the expected sensing conditions rather than with benchmark accuracy under ideal signals alone.
- Robustness properties must be treated as a design criterion before and not after embedded compression, because pruning or quantization of a disturbance-sensitive model can further reduce its practical value.
- The hardware chapter additionally shows that an adaptable research platform is not a secondary convenience, but an enabling condition for meaningful embedded-AI experimentation. The ability to exchange estimator methods, separate BMS logic from state-estimation logic, and validate new models in a controlled firmware environment is methodologically important.

- For engineering practice, this means that AI-based battery estimation should be developed together with the execution environment, measurement chain, and integration path instead of being optimized only in an offline data-analysis setting.

9.7 Integrated Design Guidelines

- Estimators should not be selected by clean-condition accuracy alone, because disturbance sensitivity and recovery behaviour strongly affect practical usefulness.
- Robustness analysis should precede aggressive compression, since a highly compressed estimator with poor disturbance tolerance offers limited deployment value.
- Quantization and pruning must be treated as architecture-dependent design choices rather than as universally transferable post-processing steps.
- Dataset quality, estimator quality, and embedded execution quality should be analysed separately so that improvements in one domain are not mistaken for improvements in another.
- Algorithmic claims gain practical relevance only when they are connected to a credible validation chain that includes sensing, firmware execution, and hardware integration.

10.1 Overall Conclusion

- The work shows that deployment-oriented battery AI emerges only when model design, robustness, embedded execution, and hardware integration are treated as one linked problem.
- Neural architectures enable powerful battery state estimation and ageing estimation, but their practical value depends on how they behave under disturbances and how efficiently they can be transferred to embedded hardware.
- Robustness benchmarking reveals that estimator quality changes fundamentally once measurement corruption, timing irregularities, and initialization mismatch are considered explicitly.
- Embedded implementation demonstrates that compression, latency, and memory demand are not secondary engineering details, but core evaluation dimensions for practical battery-management applications.
- The combination of algorithmic studies and custom hardware integration therefore leads to a unified framework for analysing battery intelligence across performance, hardware, and deployment requirements.

10.2 Boundary Conditions and Remaining Gaps

- The reported findings must be interpreted in the context of the available datasets, the selected estimator classes, and the concrete hardware environments used throughout the work.
- The general design logic of the dissertation is transferable, but the exact quantitative trade-offs between accuracy, robustness, memory demand, and latency

can shift when other cell chemistries, sensing chains, control architectures, or MCU families are considered.

- Further validation across broader datasets, longer ageing horizons, and additional embedded targets is therefore necessary to consolidate the transferability of the derived design rules.

10.3 Outlook

- Future work extends the embedded analysis by comparing pruning and quantization across a wider range of model architectures.
- Further experiments on the BMS-STEP platform can deepen the relation between estimator design, signal quality, and practical system behaviour under laboratory and hardware-in-the-loop conditions.
- Long-duration cycling and additional datasets can refine the calibration and validation of SoC and SoH methods across broader ageing states and operating conditions.
- The longer-term objective is the transfer of deployment-oriented battery AI from controlled laboratory systems toward scalable and reproducible BMS integration in larger energy-storage applications.

Literatur

- [1] Florian Rzepka, Philipp Hematty, Mano Schmitz und Julia Kowal. *Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids*. First dissertation paper. 2023 (cited on pp. 4, 13, 21–27, 29–32, 35).
- [2] Florian Rzepka und Julia Kowal. *Performance Benchmarking of Embedded Battery State Estimation: A Measurement-Only Comparison of Direct-Measurement, Hybrid, and Data-Driven Estimators*. Robustness benchmark manuscript. 2026 (cited on pp. 4, 13, 37–41, 43–46, 49, 50, 53, 55–57).
- [3] Florian Rzepka und Julia Kowal. *Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation*. Embedded AI manuscript. 2025 (cited on pp. 4, 13, 14, 59, 60, 62–67, 69, 72, 74).
- [4] Florian Rzepka und Julia Kowal. *Planned comparative study on pruning and quantization across embedded BMS model architectures*. Planned follow-up paper. 2026 (cited on p. 4).
- [5] Florian Rzepka. *D7.4 Fully programmed BMS adapted to usage behaviour and cell type*. Project deliverable, Technische Universitaet Berlin. 2024 (cited on pp. 4, 13–19).
- [6] Florian Rzepka. *D7.11 Finalised and adapted battery management system (BMS)*. Project deliverable, Technische Universitaet Berlin. 2025 (cited on pp. 4, 13–19).

Anhang **Supplementary Material**

A

This appendix contains supplementary tables, additional hardware documentation, experiment matrices, and publication-specific material that would interrupt the flow of the main text if it were placed directly in the core chapters.